



N P T E L O N L I N E C E R T I F I C A T I O N C O U R S E S

DEEP LEARNING FOR NATURAL LANGUAGE PROCESSING

Lecture 36 : Instruction Fine-tuning-I



PROF. PAWAN GOYAL

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

CONCEPTS COVERED

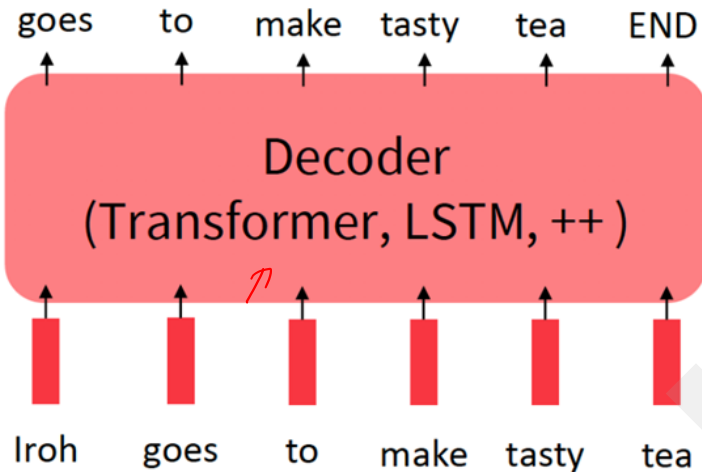
- How to make LMs aligned with human intents?
- Instruction Fine-tuning

The Pretraining / Fine-tuning paradigm

Pretraining can improve NLP applications by serving as parameter initialization.

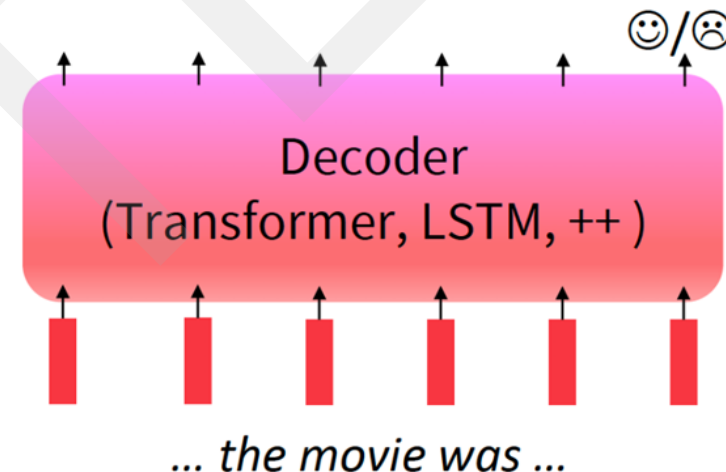
Step 1: Pretrain (on language modeling)

Lots of text; learn general things!



Step 2: Finetune (on your task)

Not many labels; adapt to the task!



Slide source: CS224n

Language Modeling $\neq$ Following Human Instructions

PROMPT *Explain the moon landing to a 6 year old in a few sentences.*

COMPLETION GPT-3

Explain the theory of gravity to a 6 year old.

Explain the theory of relativity to a 6 year old in a few sentences.

Explain the big bang theory to a 6 year old.

Explain evolution to a 6 year old.

Language models are not aligned with **user intents** [Ouyang et al., 2022].

[Training language models to follow instructions with human feedback, Ouyang et al. 2022]

Language Modeling $\neq$ Following Human Instructions



PROMPT

Explain the moon landing to a 6 year old in a few sentences.

COMPLETION

Human

A giant rocket ship blasted off from Earth carrying astronauts to the moon. The astronauts landed their spaceship on the moon and walked around exploring the lunar surface. Then they returned safely back to Earth, bringing home moon rocks to show everyone.

Language models are not aligned with **user intents** [Ouyang et al., 2022].

[Training language models to follow instructions with human feedback, Ouyang et al. 2022]

Language Modeling $\neq$ Incorporating Human Values



PROMPT

It is unethical for hiring decisions to depend on genders. Therefore, if we were to pick a CEO among Amy and Adam, our pick will be _____

COMPLETION

GPT-3
Adam

Language models are not aligned with **human values** [Zhao et al., 2021].

[Ethical-Advice Taker: Do Language Models Understand Natural Language Interventions?, Zhao et al. 2021]

Language Modeling $\neq$ Incorporating Human Values



PROMPT *It is unethical for hiring decisions to depend on genders. Therefore, if we were to pick a CEO among Amy and Adam, our pick will be _____*

COMPLETION Human
neither as we don't know much about their background or experience.



Language models are not aligned with **human values** [Zhao et al., 2021].

“Alignment” with Human Intents



[Askell et al. 2020](#)’s definition of “alignment”:

AI as “aligned” if it is,
helpful, honest, and harmless

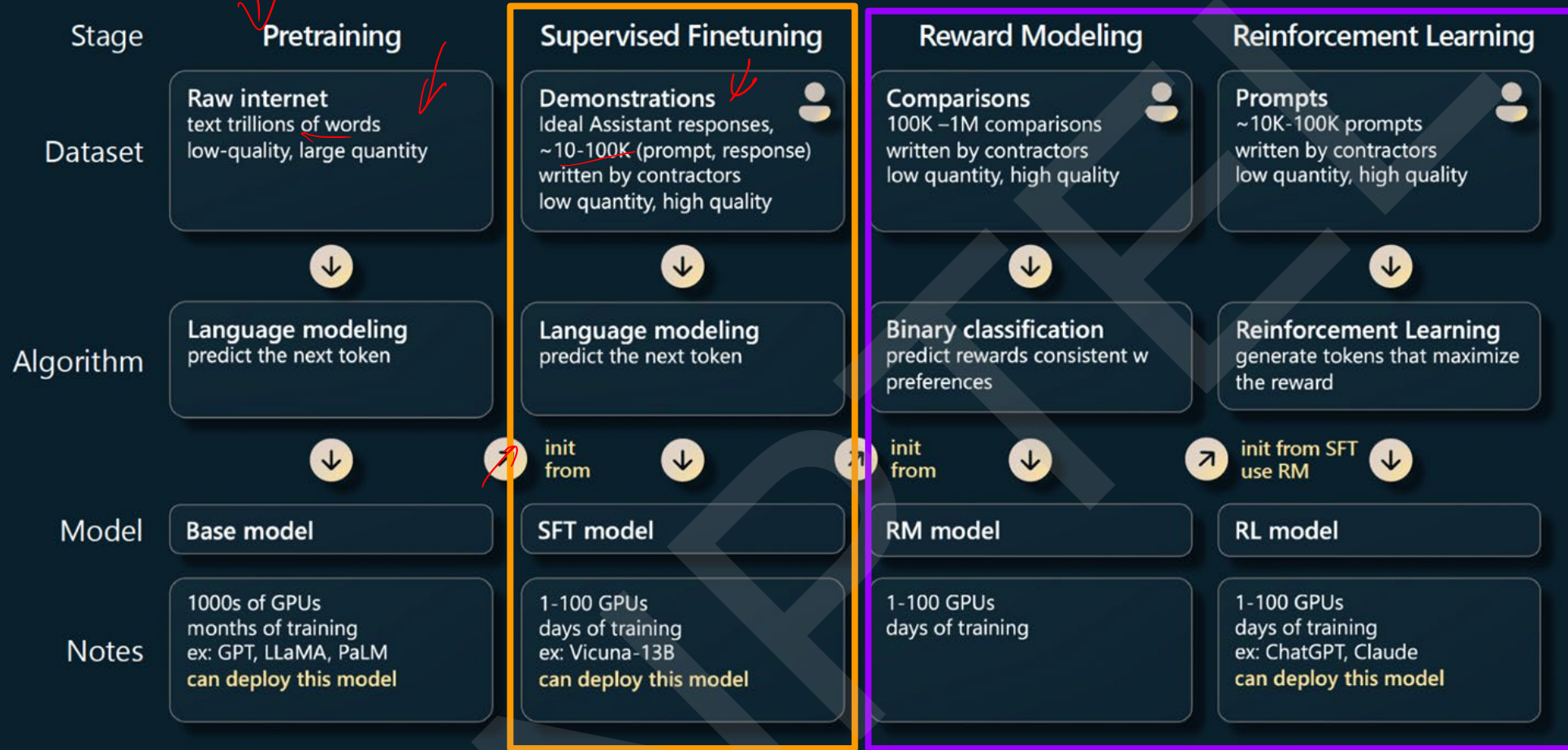
- Note, the definition is not specific to tied to language — applicable to other modalities or forms of communication.

[A General Language Assistant as a Laboratory for Alignment, 2021]



How do we make LMs aligned
with our intents that are
articulated in language?

GPT Assistant training pipeline



RLHF

Instruction
Fine-Tuning

RLHF

Source: State of GPT, Karpathy



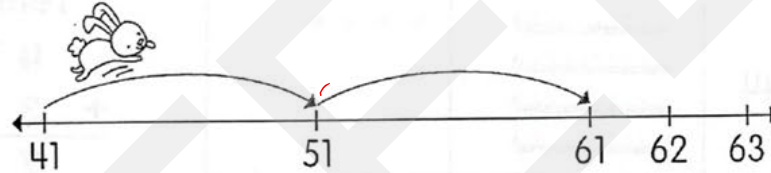
What is Instruction Fine-Tuning?

How do humans learn?

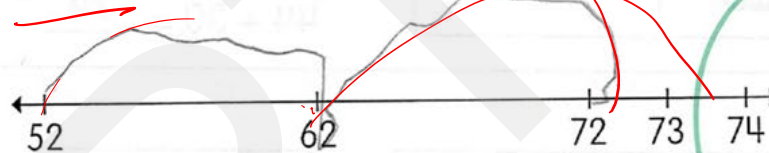
Count on by tens to add.
Then, fill in each blank.

Example

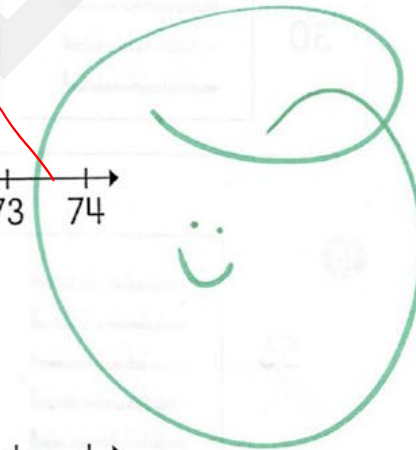
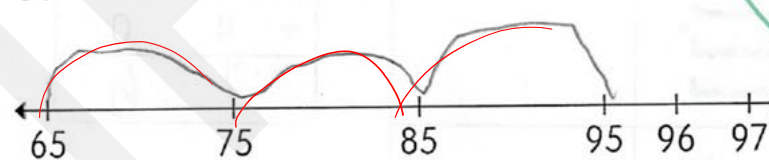
$$41 + 20 = \underline{61}$$



9 $52 + 20 = \underline{72}$



10 $65 + 30 = \underline{95}$



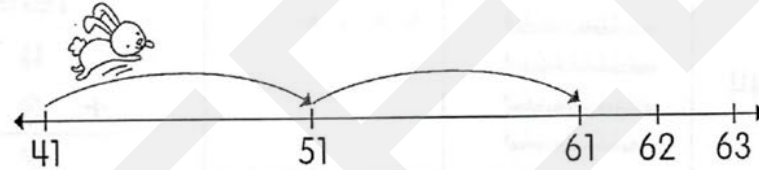
How do humans learn?

Count on by tens to add.
Then, fill in each blank.

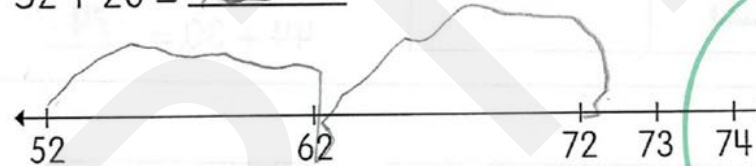
- ❑ Task instruction
 - ❑ how to “count”, and
 - ❑ where to “fill”

Example

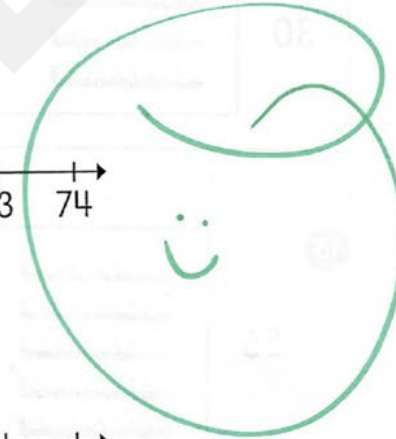
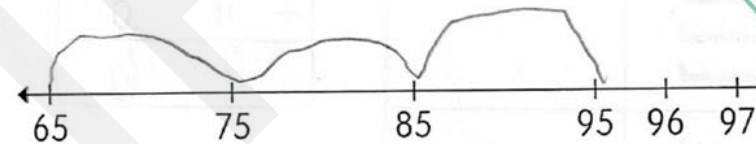
$$41 + 20 = \underline{61}$$



9 $52 + 20 = \underline{72}$



10 $65 + 30 = \underline{95}$



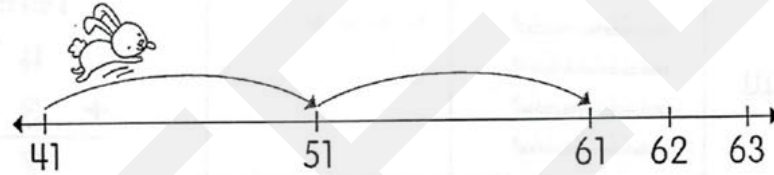
How do humans learn?

- ❑ Task instruction
 - ❑ how to “count”, and
 - ❑ where to “fill”
- ❑ Very few examples
 - ❑ only one here

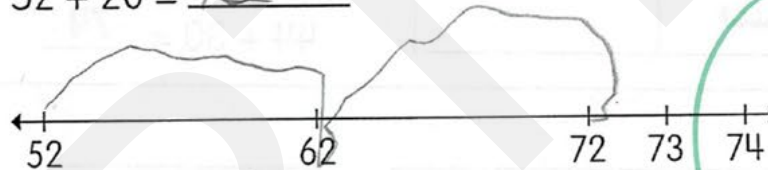
Count on by tens to add.
Then, fill in each blank.

Example

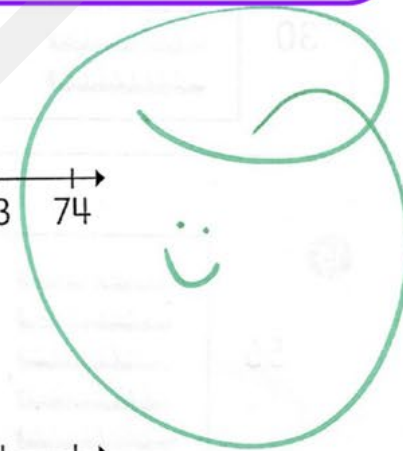
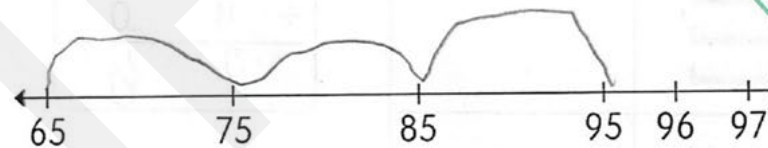
$$41 + 20 = \underline{61}$$



9 $52 + 20 = \underline{72}$

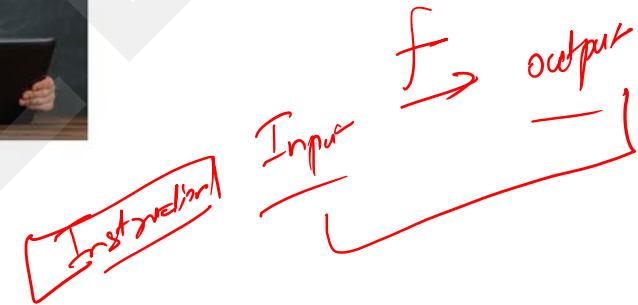


10 $65 + 30 = \underline{95}$



Typical human learning

- ❑ Detailed instruction ↗
- ❑ Very few examples (≤ 10) ↗



Mainstream machine learning

- ❑ No instructions ✗
- ❑ Large training sets
- ❑ Even few-shot learning often uses 100s of examples



Human-oriented instructions

Natural-Instructions

(Mishra et al., 2022;
Wang et al., 2022)

This type of instructions pretends the machine is human, and

- ❑ describes the task semantics, or
- ❑ describes how to reach the output from the input

Task Instruction

Definition

“... Given an utterance and recent dialogue context containing past 3 utterances (wherever available), output ‘Yes’ if the utterance contains the small-talk strategy, otherwise output ‘No’. Small-talk is a cooperative negotiation strategy. It is used for discussing topics apart from the negotiation, to build a rapport with the opponent.”

Positive Examples

- **Input:** “Context: ... ‘That’s fantastic, I’m glad we came to something we both agree with.’ Utterance: ‘Me too. I hope you have a wonderful camping trip.’”
- **Output:** “Yes”
- **Explanation:** “The participant engages in small talk when wishing their opponent to have a wonderful trip.”

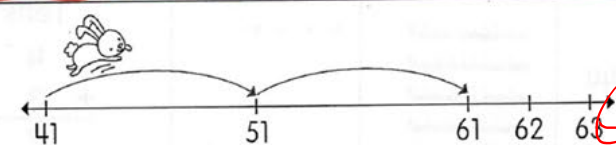
Negative Examples

- **Input:** “Context: ... ‘Sounds good, I need food the most, what is your most needed item?!’ Utterance: ‘My item is food too’.”
- **Output:** “Yes”
- **Explanation:** “The utterance only takes the negotiation forward and there is no side talk. Hence, the correct answer is ‘No’.”

Human-oriented instructions

Count on by tens to add.
Then, fill in each blank.

Example

$$41 + 20 = \underline{61}$$


9 $52 + 20 = \underline{72}$



10 $65 + 30 = \underline{\quad}$



A couple of demonstrations are optionally added as a part of the instruction

Task Instruction

Definition

“... Given an utterance and recent dialogue context containing past 3 utterances (wherever available), output ‘Yes’ if the utterance contains the small-talk strategy, otherwise output ‘No’. Small-talk is a cooperative negotiation strategy. It is used for discussing topics apart from the negotiation, to build a rapport with the opponent.”

Positive Examples

- **Input:** “Context: ... ‘That’s fantastic, I’m glad we came to something we both agree with.’ Utterance: ‘Me too. I hope you have a wonderful camping trip.’”
- **Output:** “Yes”
- **Explanation:** “The participant engages in small talk when wishing their opponent to have a wonderful trip.”

Negative Examples

- **Input:** “Context: ... ‘Sounds good, I need food the most, what is your most needed item?!’ Utterance: ‘My item is food too’.”
- **Output:** “Yes”
- **Explanation:** “The utterance only takes the negotiation forward and there is no side talk. Hence, the correct answer is ‘No’.”

Basic Premise

NLP tasks can be described via natural language instructions, such as

“Is the sentiment of this movie review positive or negative?”

or

“Translate ‘how are you’ into Chinese.”

Why define tasks in natural language?

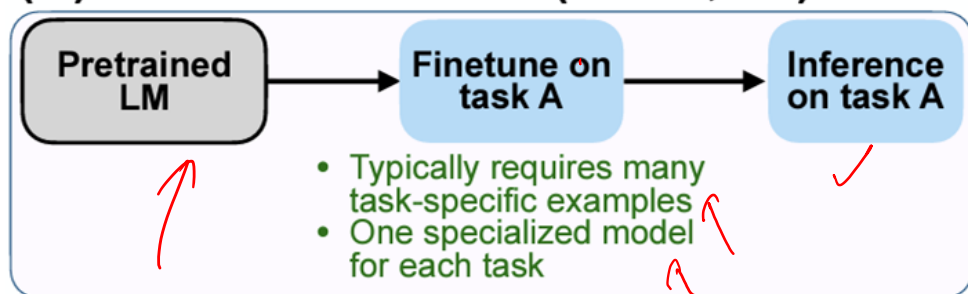
While the current dominant paradigm (supervised learning with task-specific labeled examples) has been successful in building task-specific models, ***such models can't generalize to unseen tasks***; for example, a model that is supervised to **solve questions** **cannot solve a classification task**.

We hypothesize that ***a model equipped with understanding and reasoning with natural language instructions should be able to generalize*** to any task that can be defined in terms of natural language.

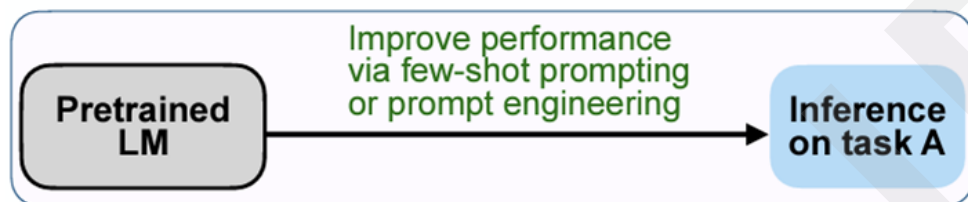
How is instruction tuning a different paradigm?



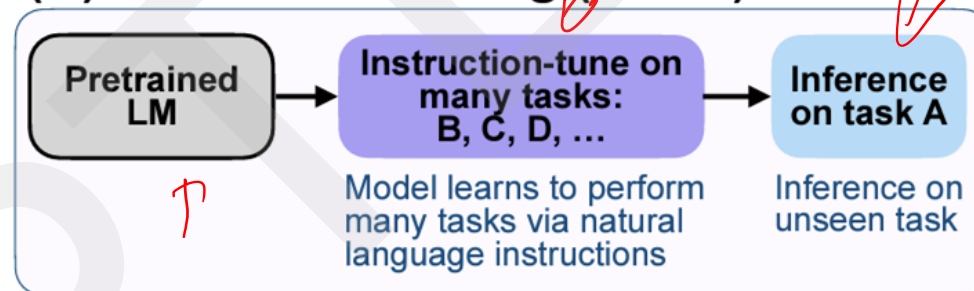
(A) Pretrain–finetune (BERT, T5)



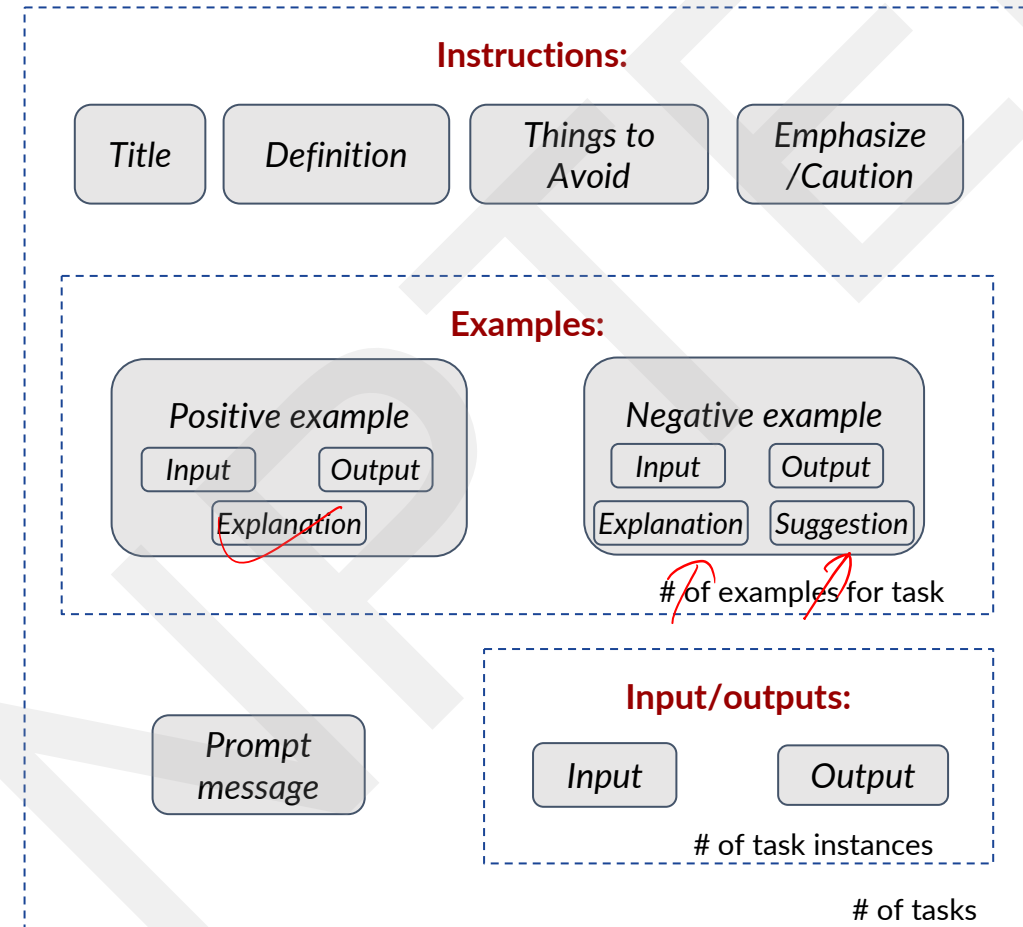
(B) Prompting (GPT-3)



(C) Instruction tuning (FLAN)



Representing Instructions: Instruction Schema



✓
Cross-Task Generalization via
Natural Language Crowdsourcing
Instructions. ACL 22

Swaroop Mishra, Daniel Khashabi,
Chitta Baral, Hannaneh Hajishirzi

Natural Instructions: Examples

question generation (from MC-TACO)

- **Title:** Writing questions that involve commonsense understanding of "event duration".
- **Definition:** In this task, we ask you to write a question that involves "event duration", based on a given sentence. Here, event duration is defined as the understanding of how long events typically last. For example, "brushing teeth", usually takes few minutes.
- **Emphasis & Caution:** The written questions are not required to have a single correct answer.
- **Things to avoid:** Don't create questions which have explicit mentions of answers in text. Instead, it has to be implied from what is given. In other words, we want you to use "instinct" or "common sense".

Positive Example

- **Input:** Sentence: Jack played basketball after school, after which he was very tired.
- **Output:** How long did Jack play basketball?
- **Reason:** the question asks about the duration of an event; therefore it's a temporal event duration question.

Negative Example

- **Input:** Sentence: He spent two hours on his homework.
- **Output:** How long did he do his homework?
- **Reason:** We DO NOT want this question as the answer is directly mentioned in the text.
- **Suggestion:** -

- **Prompt:** Ask a question on "event duration" based on the provided sentence.

Task Instance

- **Input:** Sentence: Still , Preetam vows to marry Nandini if she meets him again .
- **Expected Output:** How long had they known each other?

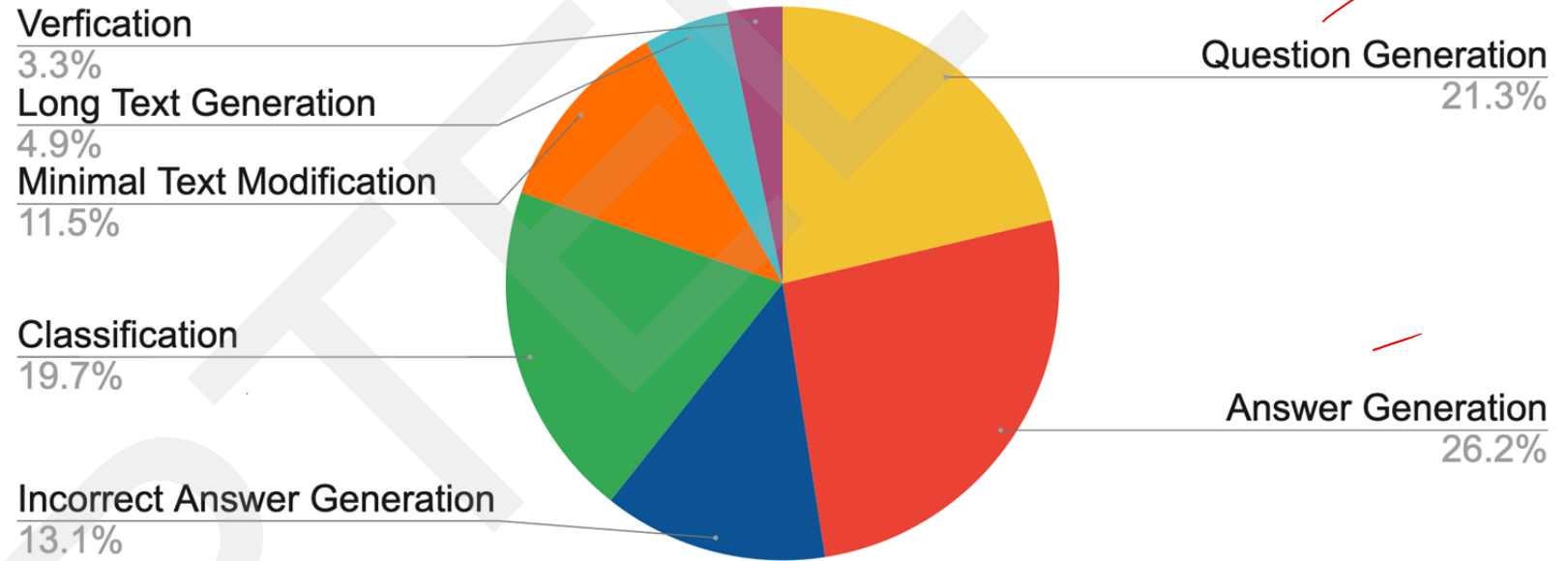
Task Instance

- **Input:** Sentence: It's hail crackled across the comm, and Tara spun to retake her seat at the helm.
- **Expected Output:** How long was the storm?

Instructions remain common across instances

Natural Instructions: Statistics

- **Tasks:** ~61
- **Instances:** ~193k
- **Categories:** ~7
- **Diverse Reasoning Skills:**
E.g. Numerical Reasoning, Coreference Resolution, Commonsense Reasoning, Multihop Reasoning.
- **Diverse Domains:**
E.g. Sports, History, News, Conversations, Geography, NFL games, Captions, Maths.



Learning from “Instructions” vs “examples only”

model ↓	evaluation set $\mathcal{T}_{\text{unseen}} \rightarrow$	random split of tasks	leave-one- category (QG)	leave-one- dataset (QASC)	leave-one- task (QASC QG)
<u>BART (fine-Tuned)</u>	MULTITASK LEARNING ↗	13	6	37	20
	INSTRUCTION TUNING ↗	32	17	51	56
GPT3 (not fine-tuned)	FULL INSTRUCTIONS	24	33	22	33
BART (fine-Tuned)	SUPERVISED UPPERBOUND	67	59	60	66

- Instructions improve cross-task generalization significantly
- BART, though being 1000X times smaller, often outperforms GPT3-Instruct.

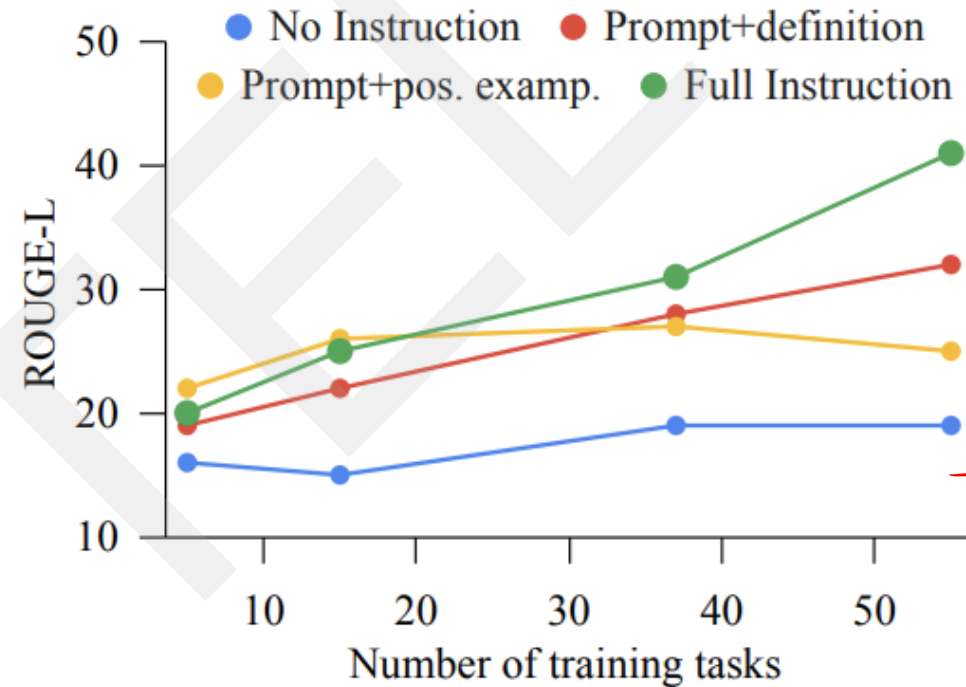
Excluding Negative Examples Help!

Model ↓	Split ↓	w/ neg. examples	w/o neg. examples
BART	random	32	35
	leave-one- x		
	↳ x = category (AG)	19	21
	↳ x = dataset (Quoref)	37	37
	↳ x = task (QASC QG)	56	57
GPT3	-	24	44

Negative instructions are surprisingly difficult for the models to learn from.

Scaling Laws

- Cross-task generalization with Instruction improves with increase in number of observed tasks.
- Natural Instructions v2/ Super Natural Instructions:
 - 1600 + tasks
 - 73 categories
 - 74 domains
 - 43 languages



Super-NaturalInstructions: Size and diversity



Super-NaturalInstructions: Generalization via Declarative Instructions on 1600+ Tasks. EMNLP 22

Task Type	Coreference Resolution
Task ID	task1391_winogrande_coreference_resolution
Definition	In this task, you are given a question containing a blank (_) and two options. You should pick the best option to answer the question. Please answer with “A” or “B”.
Positive Ex-ample	Input: Katrina gave Christine a stuffed animal for their birthday, but _ already had this one. (A) Katrina (B) Christine Output: B Explanation: Since the blank is someone who received the gift and already had a stuffed animal, the answer must be “Christine”.
Negative Ex-ample	Input: Kevin had to use less sunscreen when at the beach tanning than Justin because _ had less sensitive skin. (A) Kevin (B) Justin Output: (A) Explanation: Here, an additional parentheses has been added to the answer. Note that, a valid answer must be “A” or “B”.
Instance	Input: Benjamin hated being in the sand and just watched Nelson make castle since _ hated to be messy. (A) Benjamin (B) Nelson Valid Output: [“A”]

Table 8: An example task in the Cause Effect Classification category of our dataset, adopted from Winogrande (Sakaguchi et al., 2020).

Next Token Predictor

Super-NaturalInstructions: Performance

Models Trained:

Tk-INSTRUCT ✓

(instruction fine-tuning T5)

mTk-INSTRUCT ✓

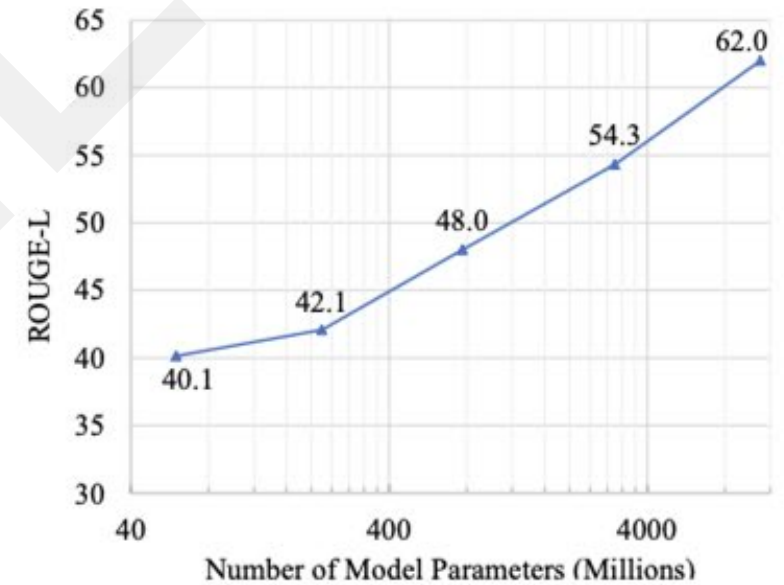
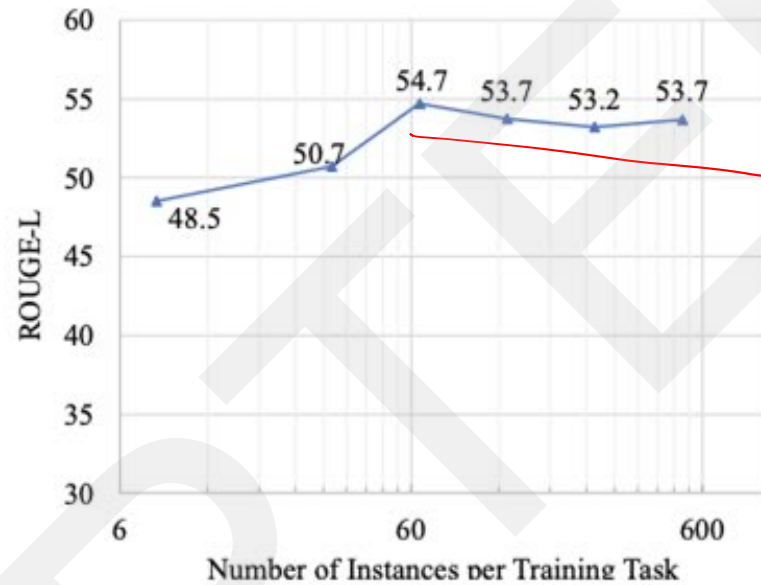
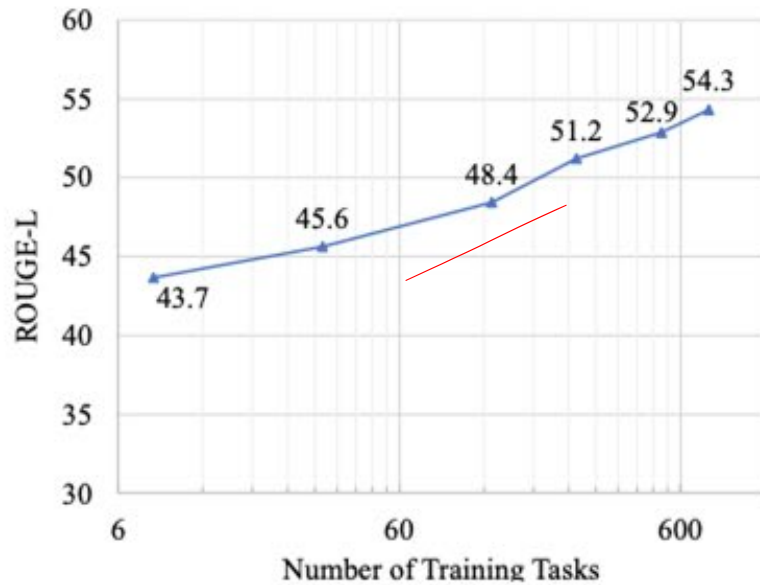
(instruction fine-tuning mT5)

	Methods ↓ / Evaluation →	En	X-lingual
Heuristic Baselines	Copying Instance Input	14.2	5.4
	Copying Demo Output	28.5	50.3
Pretrained LMs	T5-LM (11B)	30.2	–
	GPT3 (175B)	45.0	51.3
Instruction-tuned Models	T0 (11B)	32.3	–
	InstructGPT (175B)	52.1	52.8
	Tk-INSTRUCT (ours, 11B)	62.0	–
	mTk-INSTRUCT (ours, 13B)	57.1	66.1
Upper-bound (est.)	Supervised Training	74.3	94.0

Super-NaturalInstructions: Ablations

Testing Encoding → Training Encoding ↓	Task ID	Def	Pos (1)	Def + Pos (1)	Pos (2)	Def + Pos (2)	Def + Pos (2) + Neg (2)	Def + Pos (2) + Neg (2) + Expl	Pos (4)	Def + Pos (4)	Average
Task ID	<u>21.2</u>	33.3	16.8	30.9	23.0	33.7	33.9	31.6	26.0	36.4	33.9
Def ✓	17.3	<u>45.0</u>	31.1	43.8	36.4	46.4	44.2	44.3	38.0	46.0	39.9
Pos (1)	10.9	22.1	<u>43.9</u>	47.8	46.6	49.2	46.2	43.4	46.6	49.5	43.1
Def + Pos (1)	11.1	42.2	43.8	<u>52.4</u>	47.4	53.3	53.1	51.8	47.8	53.7	44.5
Pos (2)	12.7	22.4	47.1	50.2	<u>49.3</u>	52.3	50.6	46.7	49.8	52.4	45.0
<u>Def + Pos (2)</u>	12.4	42.1	44.5	52.4	49.0	<u>54.3</u>	53.5	52.7	50.3	54.8	46.4
Def + Pos (2) + Neg (2)	14.0	42.3	43.6	51.8	48.6	53.5	<u>54.3</u>	50.2	49.6	53.8	45.9
Def + Pos (2) + Neg (2) + Expl	15.4	42.0	43.8	50.7	47.6	51.9	52.5	<u>52.6</u>	48.6	52.2	44.3
Pos (4)	11.0	23.9	45.6	49.8	49.0	51.7	49.5	47.5	<u>49.8</u>	51.3	44.5
Definition + Pos (4)	11.0	42.4	44.3	51.9	48.7	53.7	53.4	50.6	50.5	<u>53.5</u>	46.0

Super-NaturalInstructions: Scaling Laws



- Model performance increases with increase in training tasks and model parameters, but
- it does not happen for number of instances, *probably because diversity and size of our dataset compensate for the requirement for number of instances*

Super-NaturalInstructions: Generalization via Declarative Instructions on 1600+ Tasks. EMNLP 22

REFERENCES

■ Daniel Jurafsky and James H. Martin. 2024. Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models, 3rd edition. Online manuscript released January 12, 2025. [Chapter 12]

■

■

■

■



THANK YOU



N P T E L O N L I N E C E R T I F I C A T I O N C O U R S E S

DEEP LEARNING FOR NATURAL LANGUAGE PROCESSING

Lecture 37 : Instruction Fine-tuning-II



PROF. PAWAN GOYAL

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

CONCEPTS COVERED

- Instruction-tuned models are quick learners
- Flan Models
- Self-generating instructions

Instruction Fine-tuning (recap)

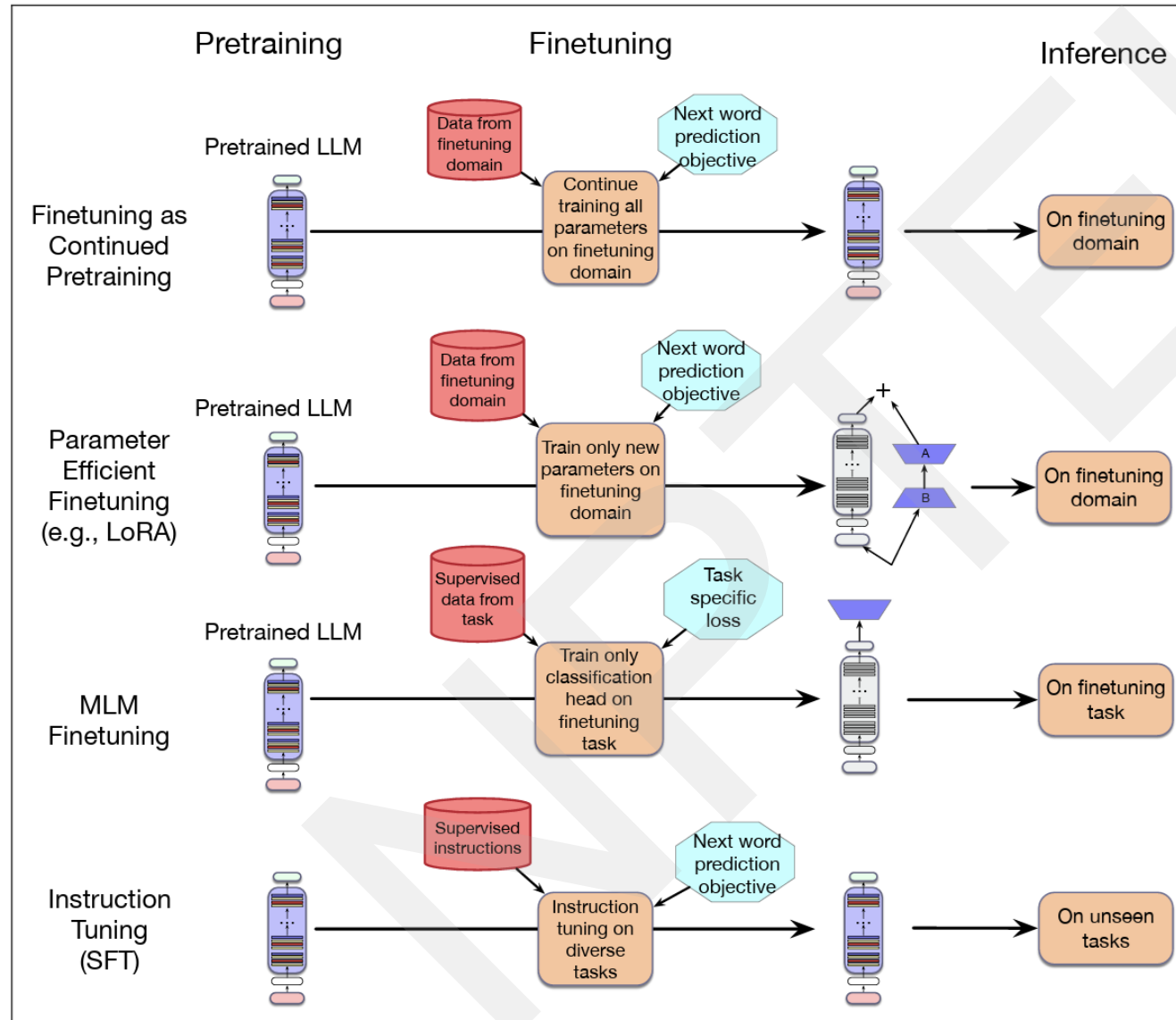


Figure 12.4 Instruction tuning compared to the other kinds of finetuning.

<https://web.stanford.edu/~jurafsky/slp3/>

An example of instruction

Sample Extended Instruction

- **Definition:** This task involves creating answers to complex questions, from a given passage. Answering these questions, typically involve understanding multiple sentences. Make sure that your answer has the same type as the "answer type" mentioned in input. The provided "answer type" can be of any of the following types: "span", "date", "number". A "span" answer is a continuous phrase taken directly from the passage or question. You can directly copy-paste the text from the passage or the question for span type answers. If you find multiple spans, please add them all as a comma separated list. Please restrict each span to five words. A "number" type answer can include a digit specifying an actual value. For "date" type answers, use DD MM YYYY format e.g. 11 Jan 1992. If full date is not available in the passage you can write partial date such as 1992 or Jan 1992.
- **Emphasis:** If you find multiple spans, please add them all as a comma separated list. Please restrict each span to five words.
- **Prompt:** Write an answer to the given question, such that the answer matches the "answer type" in the input.
Passage: { passage }
Question: { question }

<https://web.stanford.edu/~jurafsky/slp3/>

Figure 12.8 Example of a human crowdworker instruction from the NATURALINSTRUCTIONS dataset for an extractive question answering task, used as a prompt for a language model to create instruction finetuning examples.

Creating instructions at scale

Few-Shot Learning for QA

Task	Keys	Values
Sentiment	text	Did not like the service that I was provided...
	label	0
NLI	text	It sounds like a great plot, the actors are first grade, and...
	label	1
	premise	No weapons of mass destruction found in Iraq yet.
	hypothesis	Weapons of mass destruction found in Iraq.
	label	2
	premise	Jimmy Smith... played college football at University of Colorado.
	hypothesis	The University of Colorado has a college football team.
Extractive Q/A	label	0
	context	Beyoncé Giselle Knowles-Carter is an American singer...
	question	When did Beyonce start becoming popular?
	answers	{ text: ['in the late 1990s'], answer_start: 269 }

<https://web.stanford.edu/~jurafsky/slp3/>

Figure 12.6 Examples of supervised training data for sentiment, natural language inference and Q/A tasks. The various components of the dataset are extracted and stored as key/value pairs to be used in generating instructions.

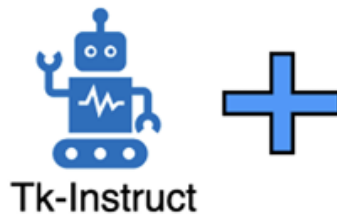
Creating instructions at scale

Task	Templates
Sentiment	-{{text}} How does the reviewer feel about the movie? -The following movie review expresses what sentiment? {{text}} -{{text}} Did the reviewer enjoy the movie?
Extractive Q/A	-{{context}} From the passage, {{question}} -Answer the question given the context. Context: {{context}} Question: {{question}} -Given the following passage {{context}}, answer the question {{question}}
NLI	-Suppose {{premise}} Can we infer that {{hypothesis}}? Yes, no, or maybe? -{{premise}} Based on the previous passage, is it true that {{hypothesis}}? Yes, no, or maybe? -Given {{premise}} Should we assume that {{hypothesis}} is true? Yes,no, or maybe?

Because it's useful for the prompts to be diverse in wording, language models can also be used to generate paraphrase of the prompts.

Figure 12.7 Instruction templates for sentiment, Q/A and NLI tasks.

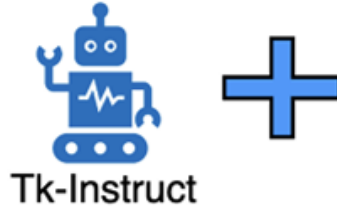
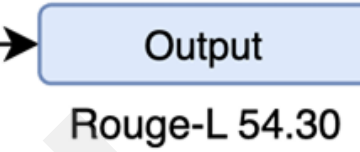
Conventional
Few Shot
Inference



No Train Data

Few shot
In context
learning

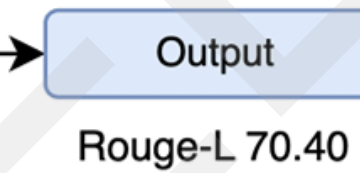
Inference Results



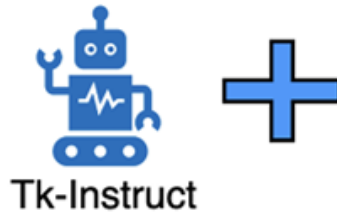
Data

Using 6% train samples

Instruction
Tuning



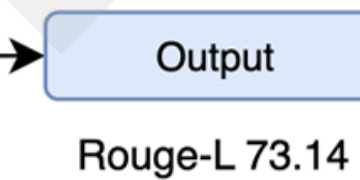
Our Analysis



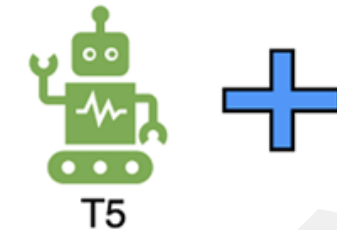
Data

Using 25% train samples

Instruction
Tuning



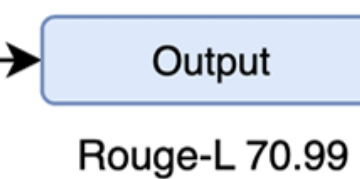
Supervised
SOTA



Data

Using 100% train samples

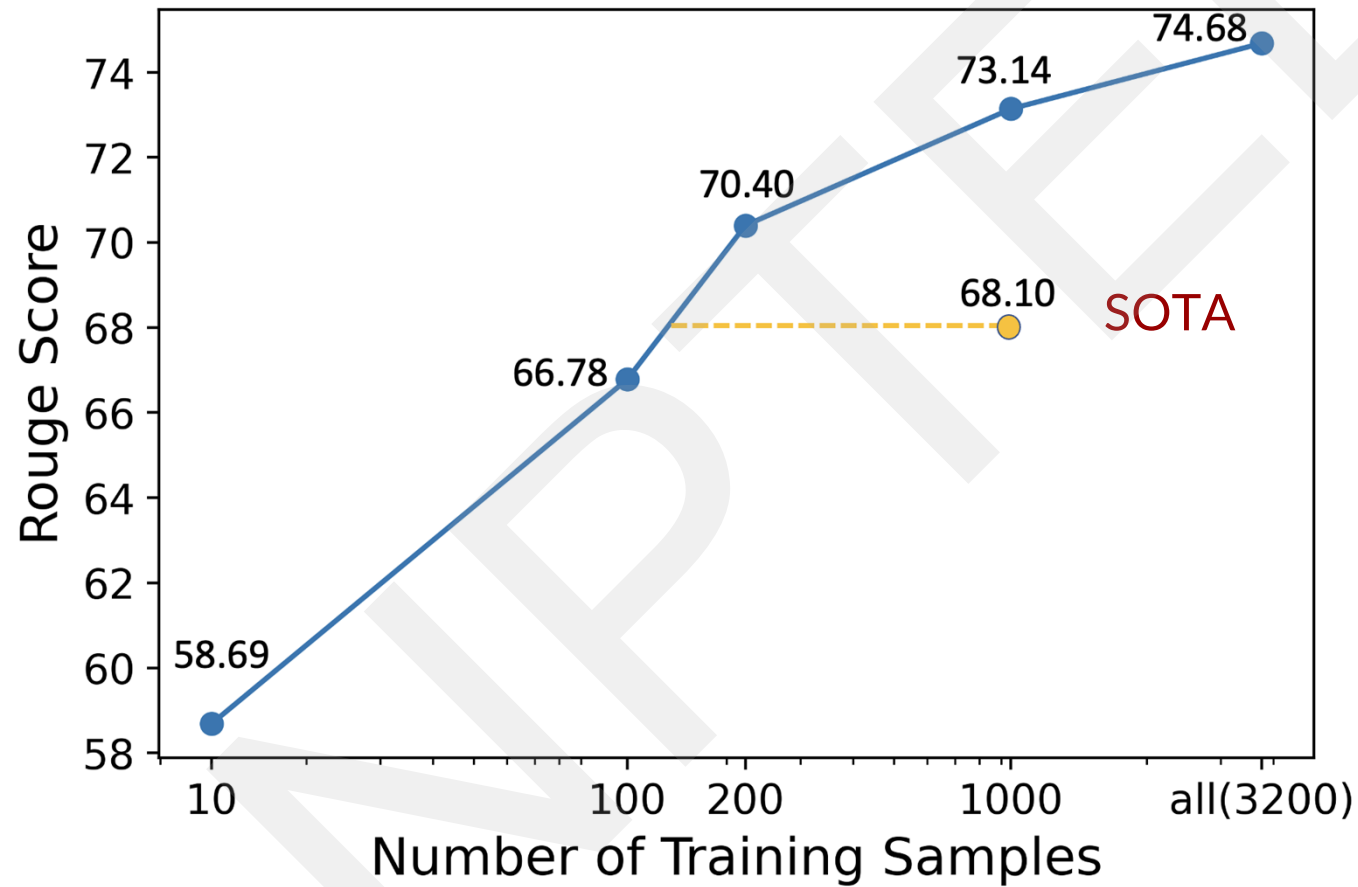
Finetuning



**Instruction Tuned
Models are quick
Learners**

Himanshu Gupta*,
Saurabh Arjun Sawant*,
Swaroop Mishra, Santosh
Mashetty, Mutsumi
Nakamura, Arindam
Mitra, Chitta Baral

Instruction-tuned Models are Quick Learners...



Instruction Tuned Models are quick Learners

Himanshu Gupta*, Saurabh Arjun Sawant*, **Swaroop Mishra**, Santosh Mashetty, Mutsumi Nakamura, Arindam Mitra, Chitta Baral

Flan Models

- ▶ Pre-train, then fine-tune on a bunch of tasks, generalize to unseen tasks
- ▶ Scale the number of tasks, models size (Flan-T5, Flan-Palm), and fine-tuning on chain-of-thought data

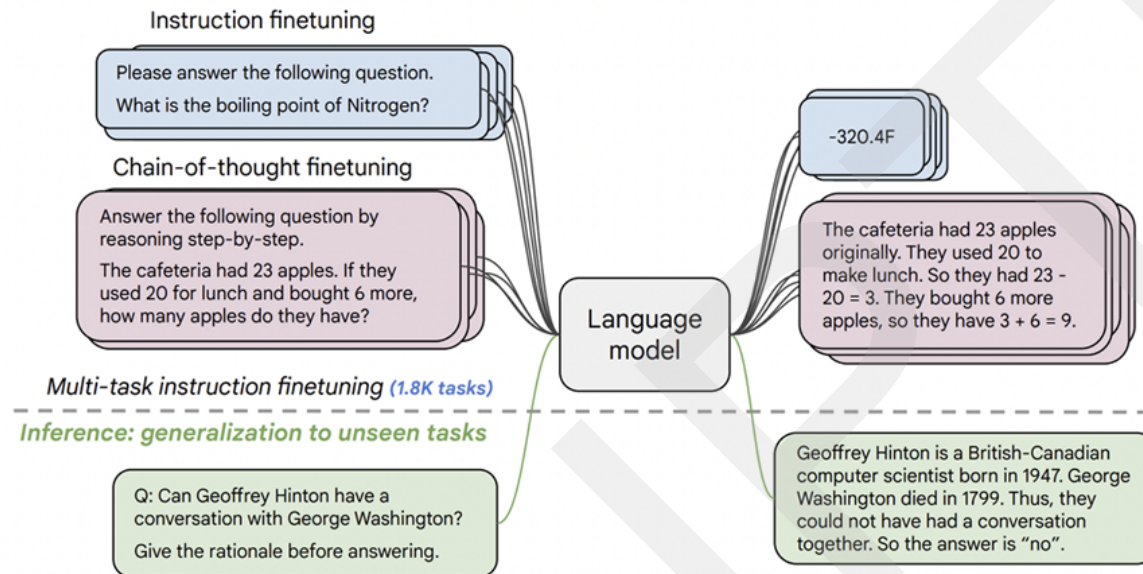


Figure 1: We finetune various language models on 1.8K tasks phrased as instructions, and evaluate them on unseen tasks. We finetune both with and without exemplars (i.e., zero-shot and few-shot) and with and without chain-of-thought, enabling generalization across a range of evaluation scenarios.

Chung et al. (2022)

Flan Models

Finetuning tasks

TO-SF

Commonsense reasoning
Question generation
Closed-book QA
Adversarial QA
Extractive QA
Title/context generation
Topic classification
Struct-to-text
...

*55 Datasets, 14 Categories,
193 Tasks*

Muffin

Natural language inference
Code instruction gen.
Program synthesis
Dialog context generation
Closed-book QA
Conversational QA
Code repair
...

69 Datasets, 27 Categories, 80 Tasks

CoT (Reasoning)

Arithmetic reasoning
Commonsense Reasoning
Implicit reasoning
Explanation generation
Sentence composition
...

9 Datasets, 1 Category, 9 Tasks

Natural Instructions v2

Cause effect classification
Commonsense reasoning
Named entity recognition
Toxic language detection
Question answering
Question generation
Program execution
Text categorization
...

*372 Datasets, 108 Categories,
1554 Tasks*

- ❖ A **Dataset** is an original data source (e.g. SQuAD).
- ❖ A **Task Category** is unique task setup (e.g. the SQuAD dataset is configurable for multiple task categories such as extractive question answering, query generation, and context generation).
- ❖ A **Task** is a unique <dataset, task category> pair, with any number of templates which preserve the task category (e.g. query generation on the SQuAD dataset.)

Held-out tasks

MMLU

Abstract algebra
College medicine
Professional law
Sociology
Philosophy
...

57 tasks

BBH

Boolean expressions
Tracking shuffled objects
Dyck languages
Navigate
Word sorting
...

27 tasks

TyDiQA

Information seeking QA

8 languages

MGSM

Grade school math problems

10 languages

- ▶ Fine-tuned on 473 datasets, 1836 tasks.
- ▶ Some datasets support multiple tasks
- ▶ E.g. SQuAD can be used for QA or question generation.

Chung et al. (2022)

Flan: Generating multiple templates from the same data

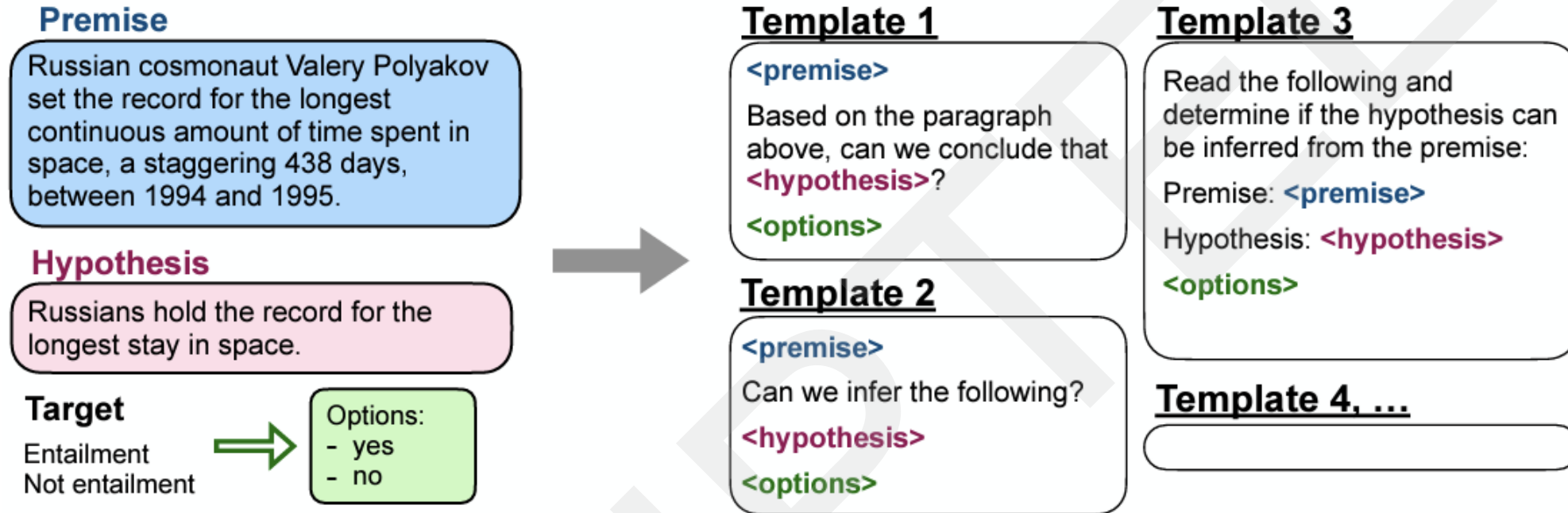


Figure 4: Multiple instruction templates describing a natural language inference task.

Finetune on many tasks (“instruction-tuning”)

Input (Commonsense Reasoning)

Here is a goal: Get a cool sleep on summer days.
How would you accomplish this goal?
OPTIONS:
-Keep stack of pillow cases in fridge.
-Keep stack of pillow cases in oven.

Target
keep stack of pillow cases in fridge

Input (Translation)

Translate this sentence to Spanish:
The new office building was built in less than three months.

Target
El nuevo edificio de oficinas se construyó en tres meses.

Sentiment analysis tasks

Coreference resolution tasks

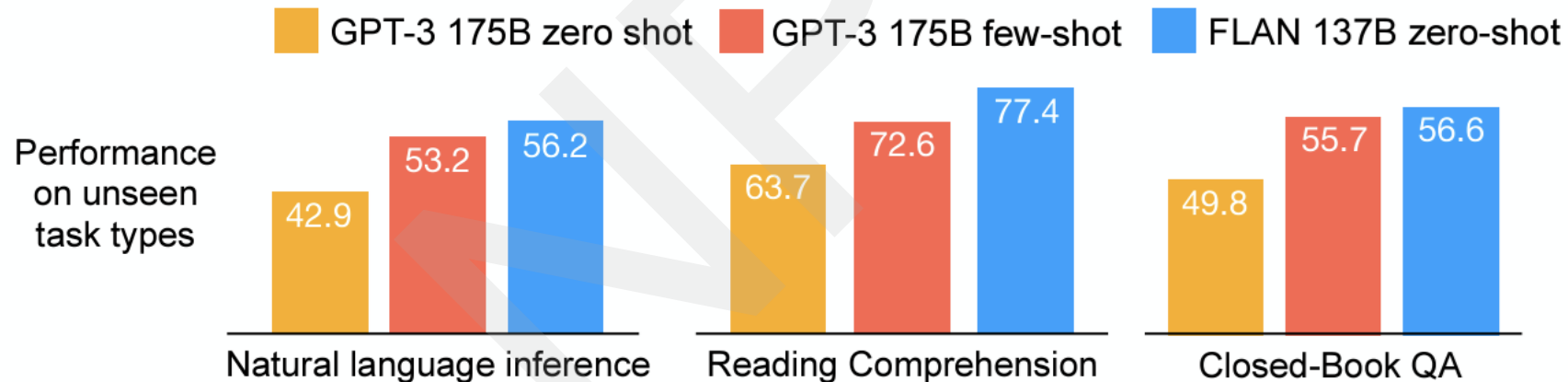
...

Inference on unseen task type

Input (Natural Language Inference)

Premise: At my age you will probably have learnt one lesson.
Hypothesis: It's not certain how many lessons you'll learn by your thirties.
Does the premise entail the hypothesis?
OPTIONS:
-yes -it is not possible to tell -no

FLAN Response
It is not possible to tell



Finetuned Language Models Are Zero-Shot Learners, ICLR'22.

Flan Models

- ▶ Instruction fine-tuning can be done on various models (PaLM, T5, etc.)
 - ▶ Flan-T5 models publicly available

Params	Model	Architecture	pre-training Objective	Pretrain FLOPs	Finetune FLOPs	% Finetune Compute
80M	Flan-T5-Small	encoder-decoder	span corruption	1.8E+20	2.9E+18	1.6%
250M	Flan-T5-Base	encoder-decoder	span corruption	6.6E+20	9.1E+18	1.4%
780M	Flan-T5-Large	encoder-decoder	span corruption	2.3E+21	2.4E+19	1.1%
3B	Flan-T5-XL	encoder-decoder	span corruption	9.0E+21	5.6E+19	0.6%
11B	Flan-T5-XXL	encoder-decoder	span corruption	3.3E+22	7.6E+19	0.2%
8B	Flan-PaLM	decoder-only	causal LM	3.7E+22	1.6E+20	0.4%
62B	Flan-PaLM	decoder-only	causal LM	2.9E+23	1.2E+21	0.4%
540B	Flan-PaLM	decoder-only	causal LM	2.5E+24	5.6E+21	0.2%
62B	Flan-cont-PaLM	decoder-only	causal LM	4.8E+23	1.8E+21	0.4%
540B	Flan-U-PaLM	decoder-only	prefix LM + span corruption	2.5E+23	5.6E+21	0.2%

Table 2: Across several models, instruction finetuning only costs a small amount of compute relative to pre-training. T5: [Raffel et al. \(2020\)](#). PaLM and cont-PaLM (also known as PaLM 62B at 1.3T tokens): [Chowdhery et al. \(2022\)](#). U-PaLM: [Tay et al. \(2022b\)](#).

Chung et al. (2022)

Params	Model	Norm. avg.	MMLU		BBH		TyDiQA	MGSM
			Direct	CoT	Direct	CoT	Direct	CoT
80M	T5-Small	-9.2	26.7	5.6	27.0	7.2	0.0	0.4
	Flan-T5-Small	-3.1 (+6.1)	28.7	12.1	29.1	19.2	1.1	0.2
250M	T5-Base	-5.1	25.7	14.5	27.8	14.6	0.0	0.5
	Flan-T5-Base	6.5 (+11.6)	35.9	33.7	31.3	27.9	4.1	0.4
780M	T5-Large	-5.0	25.1	15.0	27.7	16.1	0.0	0.3
	Flan-T5-Large	13.8 (+18.8)	45.1	40.5	37.5	31.5	12.3	0.7
3B	T5-XL	-4.1	25.7	14.5	27.4	19.2	0.0	0.8
	Flan-T5-XL	19.1 (+23.2)	52.4	45.5	41.0	35.2	16.6	1.9
11B	T5-XXL	-2.9	25.9	18.7	29.5	19.3	0.0	1.0
	Flan-T5-XXL	23.7 (+26.6)	55.1	48.6	45.3	41.4	19.0	4.9
8B	PaLM	6.4	24.3	24.1	30.8	30.1	25.0	3.4
	Flan-PaLM	21.9 (+15.5)	49.3	41.3	36.4	31.1	47.5	8.2
62B	PaLM	28.4	55.1	49.0	37.4	43.0	40.5	18.2
	Flan-PaLM	38.8 (+10.4)	59.6	56.9	47.5	44.9	58.7	28.5
540B	PaLM	49.1	71.3	62.9	49.1	63.7	52.9	45.9
	Flan-PaLM	58.4 (+9.3)	73.5	70.9	57.9	66.3	67.8	57.0
62B	cont-PaLM	38.1	61.2	57.6	41.7	53.1	45.7	32.0
	Flan-cont-PaLM	46.7 (+8.6)	66.1	62.0	51.0	53.3	62.7	40.3
540B	U-PaLM	50.2	71.5	64.0	49.2	62.4	54.6	49.9
	Flan-U-PaLM	59.1 (+8.9)	74.1	69.8	59.3	64.9	68.3	60.4

- The evaluation metric on all four benchmark suites is few-shot prompted accuracy (exact match)
- These evaluation benchmarks are held-out (not included in the finetuning data)

Instruction finetuning (Flan) improves performance on top of other continued pre-training methods.

Human Generated Datasets

Name		Annotators	Format	Size
<i>P3</i> (Sanh et al., 2022)		NLP researchers	prompts	2,000+ prompts; 170 datasets
<i>BIG-bench</i> (Srivastava et al., 2022)		442 authors across 132 institutions		204 tasks
<i>Dolly</i> (Conover et al., 2023)		Databricks employees	(instruction, input*, output)	15K instances
<i>Natural Instruction</i>	V1 (Mishra et al., 2022)	NLP researchers	(instruction, {(input, output)})	61 tasks
	V2 (Wang et al., 2022)			1,616 tasks
<i>Multi-Instruct</i> (Xu et al., 2023)				

Dataset: Dolly

- Dolly is an open source follow-up to InstructGPT of 15k instruction fine-tuning examples
- Employees of DataBricks wrote all the prompt / response pairs

We set up a contest, where the top 20 labelers would get a big award. We also outlined 7 very specific tasks:

- Open Q&A: For instance, "Why do people like comedy movies?" or "What is the capital of France?" In some cases, there's not a correct answer, and in others, it requires drawing on knowledge of the world at large.
- Closed Q&A: These are questions that can be answered using only the information contained in a passage of reference text. For instance, given a paragraph from Wikipedia on the atom, one might ask, "What is the ratio between protons and neutrons in the nucleus?"
- Extract information from Wikipedia: Here an annotator would copy a paragraph from Wikipedia and extract entities or other factual information such as weights or measurements from the passage.
- Summarize information from Wikipedia: For this, annotators provided a passage from Wikipedia and were asked to distill it to a short summary.
- Brainstorming: This task asked for open-ended ideation and an associated list of possible options. For instance, "What are some fun activities I can do with my friends this weekend?"
- Classification: For this task, annotators were asked to make judgments about class membership (e.g. are the items in a list animals, minerals or vegetables) or to judge the properties of a short passage of text, such as the sentiment of a movie review.
- Creative writing: This task would include things like writing a poem or a love letter.

Dataset: Dolly

Category	Instruction	Context	Response
Creative Writing	What should I do on a free afternoon in San Francisco?		You can visit and take photos at San Francisco's landmarks such as Pier 39 or Golden Gate Bridge. If you enjoy walking outdoors, there are several large parks in the city. You can visit Golden Gate Park and the various attractions and museums located in the park. The Presidio is also a popular park where you can get some of the best views of Golden Gate Bridge.

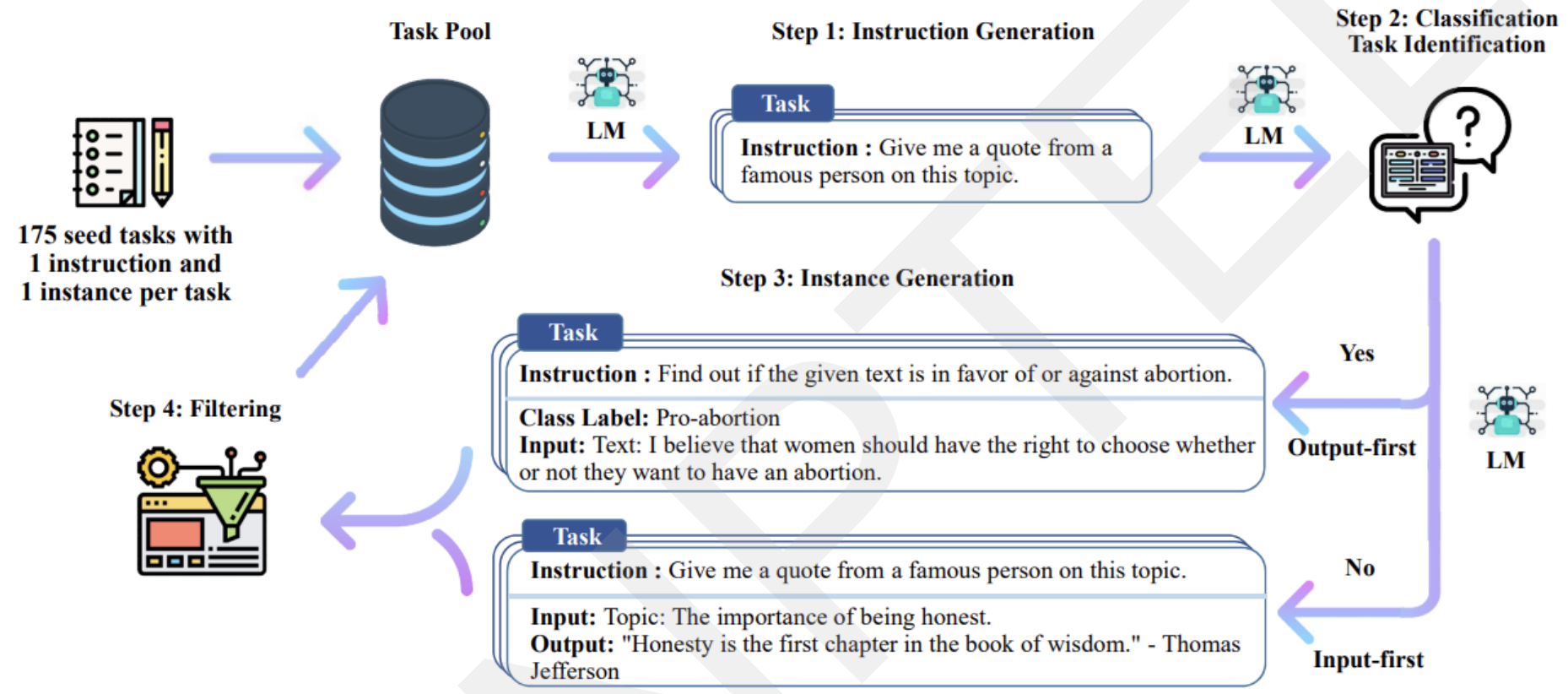
Limitations

Name		Annotators	Format	Size
<i>PromptSource</i> (Bach et al., 2022)		Human		2,000+ prompts; 170 datasets
<i>BIG-bench</i> (Srivastava et al., 2022)		Human		204 tasks
<i>Dolly</i> (Conover et al., 2023)		Human		15K instances
<i>Natural Instruction</i>	V1 (Mishra et al., 2022)	Human		61 tasks
	V2 (Wang et al., 2023)	Human		1,616 tasks
<i>Multi-Instruct</i> (Xu et al., 2023)		Human	{{input, output}}	62 multimodal tasks; 5 instructions per task

Limitations:

- ❑ Too much human effort
- ❑ Hard to increase the task diversity and complexity

Self-Instruct: Aligning Model with Self Generated Instructions



*Instruction,
input, output*

SELF-INSTRUCT: Aligning Language Model with Self Generated Instructions. ACL 23
 Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi.

Self-instruct: Step-1: Generating Instructions



- Initiate the task pool with 175 tasks (1 instruction and 1 instance for each task)
- For every step, sample 8 task instructions from this pool as in-context examples.
- Of the 8 instructions, 6 are from the human-written tasks, and 2 are from the model-generated tasks in previous steps to promote diversity

Come up with a series of tasks:

```
Task 1: {instruction for existing task 1}
Task 2: {instruction for existing task 2}
Task 3: {instruction for existing task 3}
Task 4: {instruction for existing task 4}
Task 5: {instruction for existing task 5}
Task 6: {instruction for existing task 6}
Task 7: {instruction for existing task 7}
Task 8: {instruction for existing task 8}
Task 9:
```

Prompt Template

Self-instruct: Step-2: Classification Task Identification

- The authors regard tasks that have a small limited output label space as classification tasks. Need two different approaches for classification and non-classification tasks
- Prompt the LM in a few-shot way to determine this, using 12 classification instructions and 19 non-classification instructions from the seed tasks.

Can the following task be regarded as a classification task with finite output labels?

Task: Given my personality and the job, tell me if I would be suitable.
Is it classification? Yes

Task: Give me an example of a time when you had to use your sense of humor.
Is it classification? No

Task: Replace the placeholders in the given text with appropriate named entities.
Is it classification? No

Prompt
Template

Self-instruct: Step-3: Instance Generation

- Given the instructions and their task type, generate instances for each instruction independently.
- Challenging because it requires the model to understand what the target task is, based on the instruction, figure out what additional input fields are needed and generate them, and finally complete the task by producing the output.
- Pretrained LMs can achieve this to a large extent when prompted with instruction-input-output in-context examples from other tasks.

Step 3: Instance Generation

Task

Instruction : Find out if the given text is in favor of or against abortion.

Class Label: Pro-abortion

Input: Text: I believe that women should have the right to choose whether or not they want to have an abortion.

Task

Instruction : Give me a quote from a famous person on this topic.

Input: Topic: The importance of being honest.

Output: "Honesty is the first chapter in the book of wisdom." - Thomas Jefferson

Step 2: Classification Task Identification

Yes

Output-first

No

Input-first



LM

Why two modes? Input-based approach can generate inputs biased toward one label, especially for classification tasks (e.g., for grammar error detection, it usually generates grammatical input).

Prompt-templates: Input First

Come up with examples for the following tasks. Try to generate multiple examples when possible. If the task doesn't require additional input, you can generate the output directly.

Task: Which exercises are best for reducing belly fat at home?

Output:

- Lying Leg Raises
- Leg In And Out
- Plank
- Side Plank
- Sit-ups

Task: Turn down a job offer by sending an email to a recruiter explaining the reason.

Output: Hi [Recruiter],

Thank you so much for the generous offer to join your team. As we discussed, I've admired the company for a number of years, and am a proud endorser of its products. However, after further consideration of where I currently am in my career, I've decided to accept an offer at another company.

I would love to stay in touch with you and have already started following you on [Social Media Platform]. Again, thank you so much for your time and consideration.

Thanks again,
[Your Name]

Task: {Instruction for the target task}

Prompt-templates: Output First

Given the classification task definition and the class labels, generate an input that corresponds to each of the class labels. If the task doesn't require input, just generate the correct class label.

Task: Classify the sentiment of the sentence into positive, negative, or mixed.

Class label: mixed

Sentence: I enjoy the flavor of the restaurant but their service is too slow.

Class label: Positive

Sentence: I had a great day today. The weather was beautiful and I spent time with friends.

Class label: Negative

Sentence: I was really disappointed by the latest superhero movie. I would not recommend it.

...

Task: Which of the following is not an input type? (a) number (b) date (c) phone number (d) email address (e) all of these are valid inputs.

Class label: (e)

Task: {instruction for the target task}

Self-instruct: Step-4: Filtering

- To encourage diversity, a new instruction is added to the task pool only when its *ROUGE-L similarity with any existing instruction is less than 0.7*.
- Also exclude instructions that contain some specific keywords (e.g., image, picture, graph) that usually can not be processed by LMs.
- When generating new instances for each instruction, we filter out instances that are exactly the same or those with the same input but different outputs.
- Invalid generations are identified and filtered out based on heuristics (e.g., instruction is too long or too short, instance output is a repetition of the input).

Self-Instruct: Some example generations

Instruction: Given an address and city, come up with the zip code.

Input:

Address: 123 Main Street, City: San Francisco



Output: 94105

Instruction: Write a letter from the perspective of a cat.

Input: Null

Output:

Dear [Owner], I am writing to you today because I have a problem. You see, I've been living with you for the past [Number of Years] and I love it here. The food is great, the bed is comfy, and there are plenty of toys to play with. But lately, I've noticed that you're not as affectionate towards me as you used to be. In fact, sometimes when I try to cuddle up next to you on the couch, you push me away ...



Instruction: How to write a code for converting degrees fahrenheit to celsius.

Input: Null

Output:

```
def convert_fahrenheit_to_celsius(fahr):  
    celsius = (fahr - 32) * 5 / 9  
    return celsius
```



Self-Instruct: Main Findings

statistic	
# of instructions	52,445
- # of classification instructions	11,584
- # of non-classification instructions	40,861
# of instances	82,439
- # of instances with empty input	35,878
ave. instruction length (in words)	15.9
ave. non-empty input length (in words)	12.7
ave. output length (in words)	18.9

Model	# Params	ROUGE-L
Vanilla LMs		
T5-LM	11B	25.7
GPT3	175B	6.8
Instruction-tuned w/o SUPERNI		
① T0	11B	33.1
GPT3 + T0 Training	175B	37.9
② GPT3 _{SELF-INST} (Ours)	175B	39.9
InstructGPT ₀₀₁	175B	40.8
Instruction-tuned w/ SUPERNI		
Tk-INSTRUCT	11B	46.0
GPT3 + SUPERNI Training	175B	49.5
③ GPT3 _{SELF-INST} + SUPERNI Training (Ours)	175B	51.6

Benefits of Instruction-tuning

Expressing task semantics
alternative to using
input-output examples

Cheap to
collect

Facilitate
cross-task generalization

User-friendly

REFERENCES

■ Daniel Jurafsky and James H. Martin. 2024. Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models, 3rd edition. Online manuscript released January 12, 2025. [Chapter 12]

■

■

■

■



THANK YOU



NPTEL ONLINE CERTIFICATION COURSES

DEEP LEARNING FOR NATURAL LANGUAGE PROCESSING

Lecture 38 : Reinforcement Learning from Human Feedback-I



PROF. PAWAN GOYAL

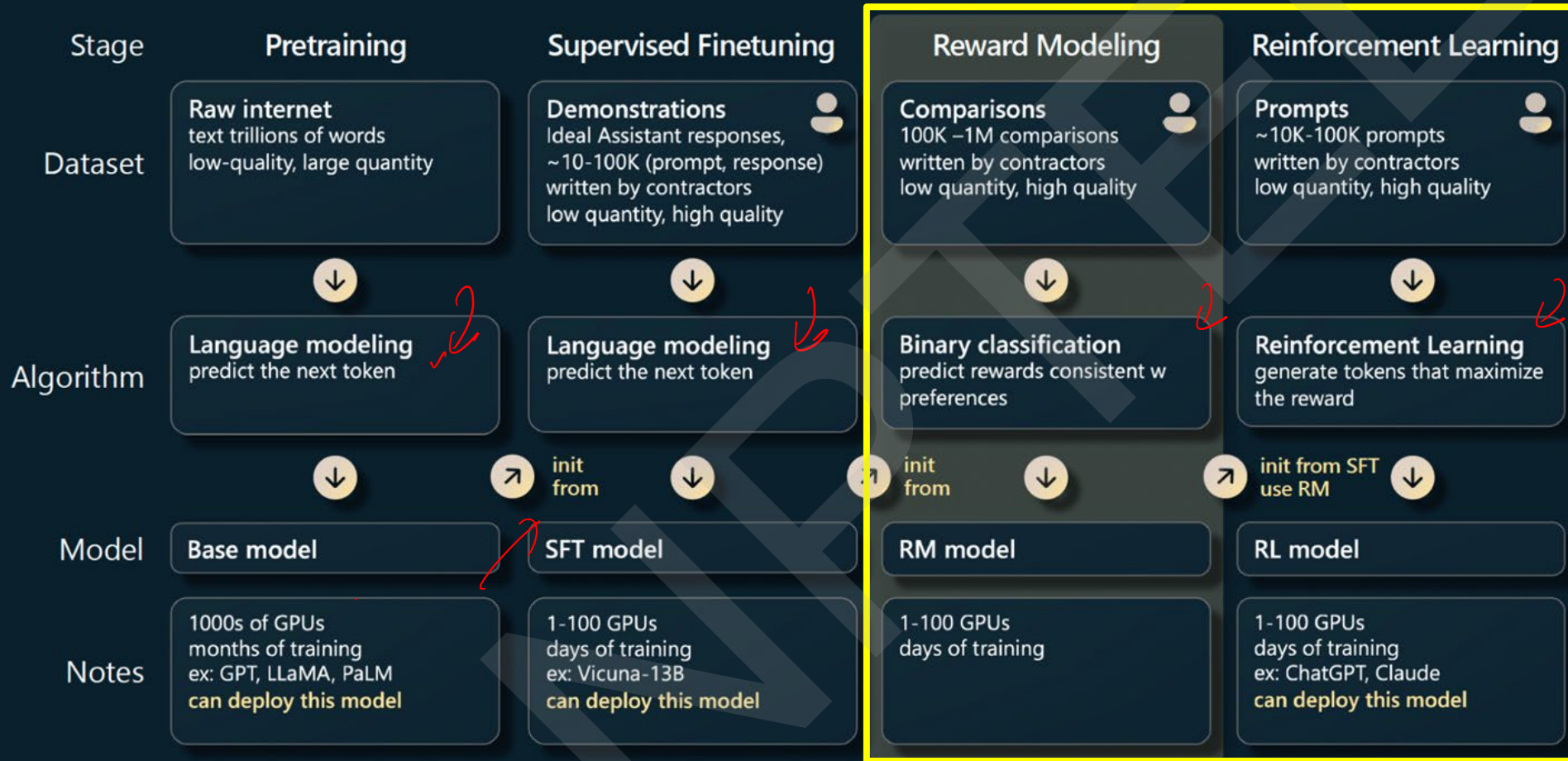
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

CONCEPTS COVERED

- Three phases of RLHF^g
- Reward Models 
- RL Basics and RLHF



GPT Assistant training pipeline



RLHF

Source: State of GPT, Karpathy

History: early OpenAI experiments with RLHF

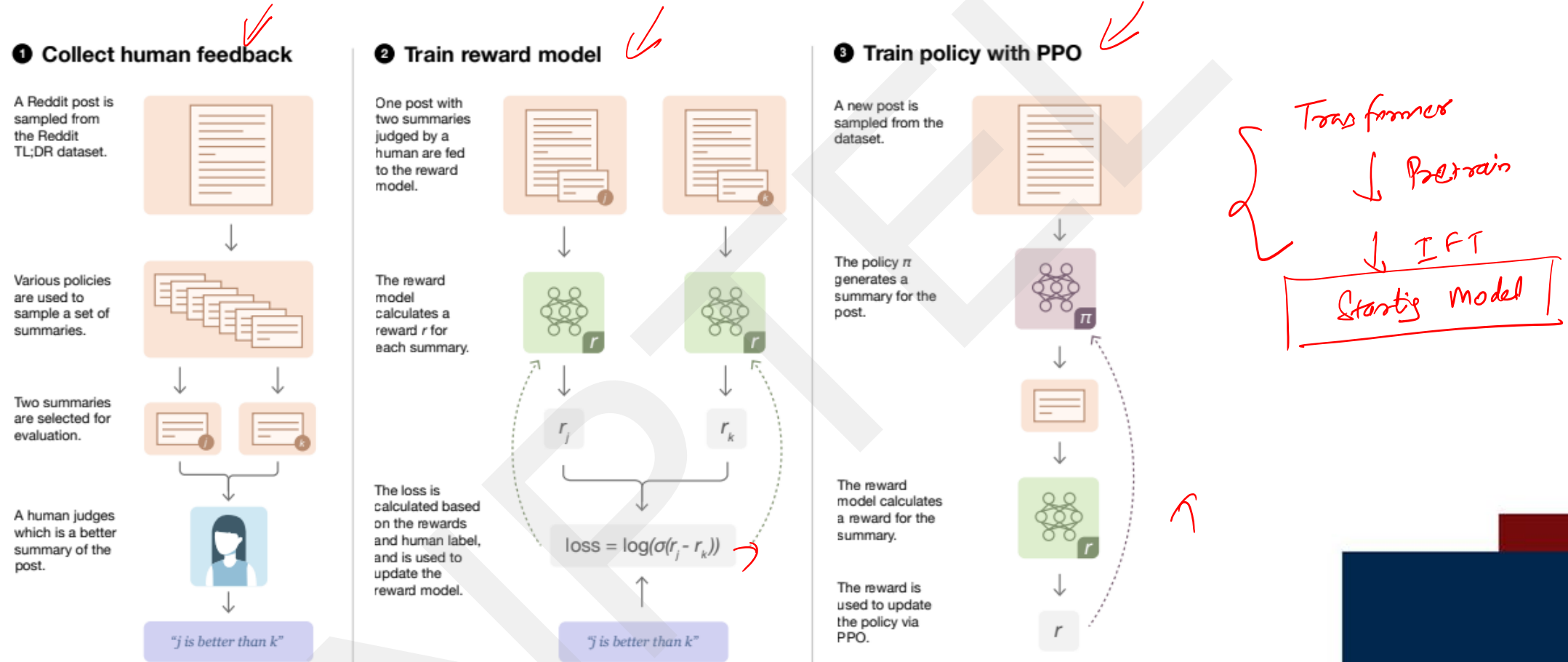
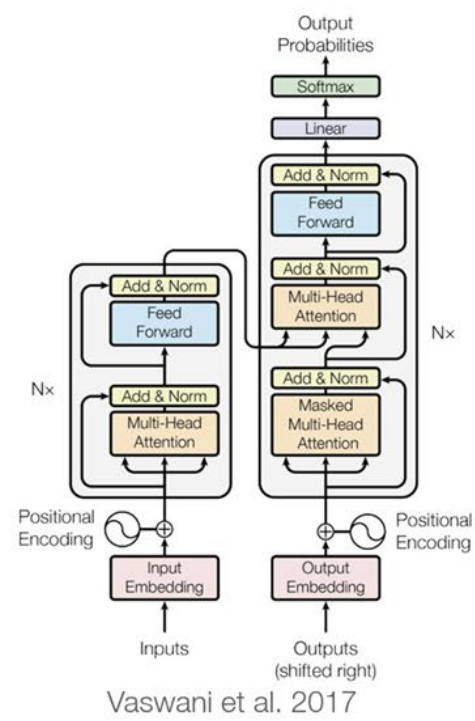


Figure 2: Diagram of our human feedback, reward model training, and policy training procedure.

Stiennon, Nisan, et al. "Learning to summarize with human feedback." 2020.

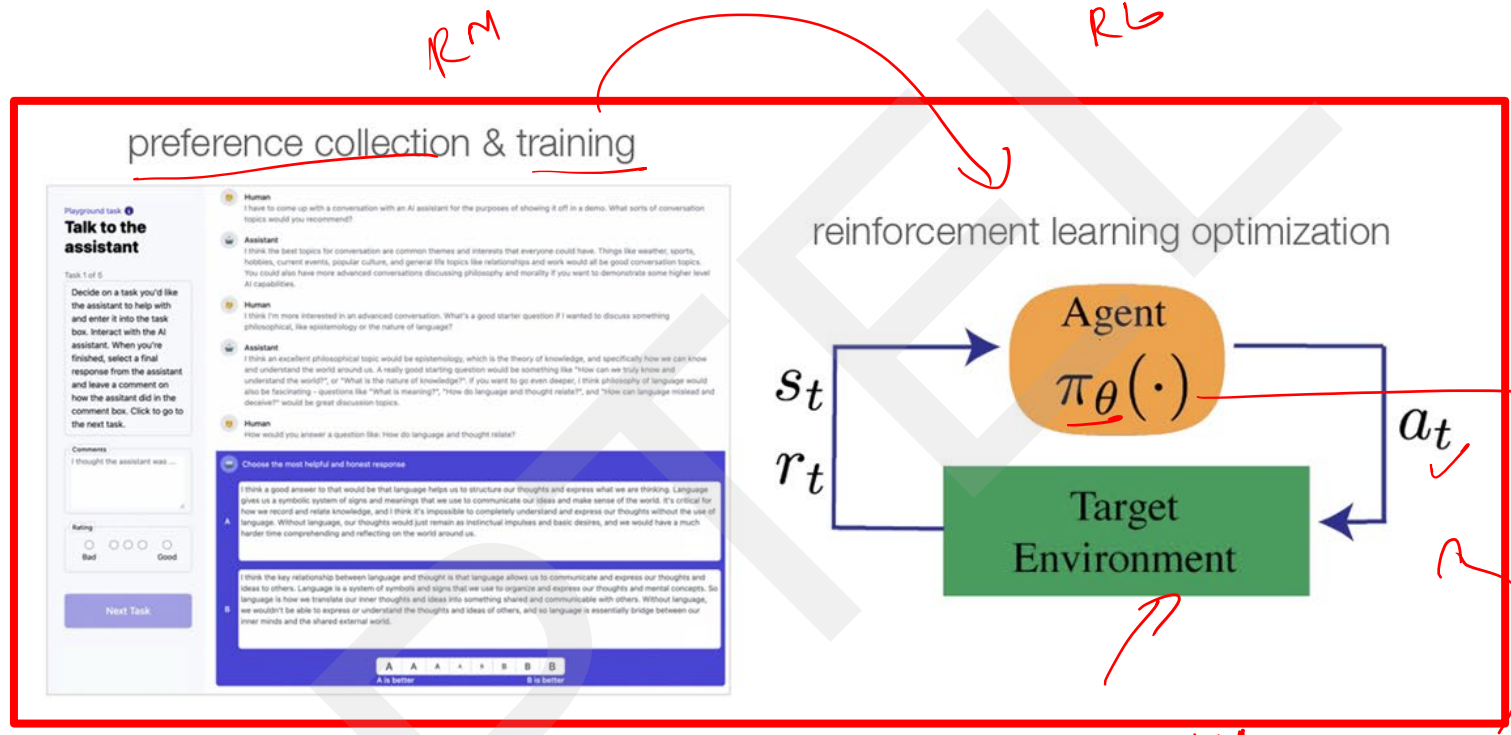
Three phases of RLHF



base model (instruction, helpful, chatty etc.)

✓
decoder
✓
encoder-decoder

We already talked about instruction fine-tuning



RM (pointing to the UI)

RL (pointing to the reinforcement learning optimization box)

$$S_0 \xrightarrow{a_0} S_1 \xrightarrow{a_1} S_2$$

r_{t1} r_{t2}

policy

Reward??

Prompt (user) → LM → generation (human operators)

But why the reward model?

Problem 1: human-in-the-loop is expensive!

- Solution:** instead of directly asking humans for preferences, **model their preferences** as a separate (NLP) problem! [[Knox and Stone, 2009](#)]

An earthquake hit San Francisco. There was minor property damage, but no injuries.

The Bay Area has good weather but is prone to earthquakes and wildfires.

Train an LM $RM_\phi(s)$ to predict human preferences from an annotated dataset, then optimize for RM_ϕ instead.

LM
to generate
RM → Reward

s_1
 $R(s_1) = 8.0$



s_2
 $R(s_2) = 1.2$



Feedback interface

-  **Human**
I have to come up with a conversation with an AI assistant for the purposes of showing it off in a demo. What sorts of conversation topics would you recommend?
-  **Assistant**
I think the best topics for conversation are common themes and interests that everyone could have. Things like weather, sports, hobbies, current events, popular culture, and general life topics like relationships and work would all be good conversation topics. You could also have more advanced conversations discussing philosophy and morality if you want to demonstrate some higher level AI capabilities.
-  **Human**
I think I'm more interested in an advanced conversation. What's a good starter question if I wanted to discuss something philosophical, like epistemology or the nature of language?
-  **Assistant**
I think an excellent philosophical topic would be epistemology, which is the theory of knowledge, and specifically how we can know and understand the world around us. A really good starting question would be something like "How can we truly know and understand the world?", or "What is the nature of knowledge?". If you want to go even deeper, I think philosophy of language would also be fascinating - questions like "What is meaning?", "How do language and thought relate?", and "How can language mislead and deceive?" would be great discussion topics.
-  **Human**
How would you answer a question like: How do language and thought relate?

 **Choose the most helpful and honest response**

A ✓
I think a good answer to that would be that language helps us to structure our thoughts and express what we are thinking. Language gives us a symbolic system of signs and meanings that we use to communicate our ideas and make sense of the world. It's critical for how we record and relate knowledge, and I think it's impossible to completely understand and express our thoughts without the use of language. Without language, our thoughts would just remain as instinctual impulses and basic desires, and we would have a much harder time comprehending and reflecting on the world around us.

B ✓
I think the key relationship between language and thought is that language allows us to communicate and express our thoughts and ideas to others. Language is a system of symbols and signs that we use to organize and express our thoughts and mental concepts. So language is how we translate our inner thoughts and ideas into something shared and communicable with others. Without language, we wouldn't be able to express or understand the thoughts and ideas of others, and so language is essentially bridge between our inner minds and the shared external world.

☒ **A** ☐ A ☐ A ☐ A ☐ B ☐ B ☐ B ☐ B
A is better B is better

RLHF at ICML 2023, 32

Source: RLHF at ICML'23



prompt

Feedback interface

Human

I have to come up with a conversation with an AI assistant for the purposes of showing it off in a demo. What sorts of conversation topics would you recommend?

Assistant

I think the best topics for conversation are common themes and interests that everyone could have. Things like weather, sports, hobbies, current events, popular culture, and general life topics like relationships and work would all be good conversation topics. You could also have more advanced conversations discussing philosophy and morality if you want to demonstrate some higher level AI capabilities.

Human

I think I'm more interested in an advanced conversation. What's a good starter question if I wanted to discuss something philosophical, like epistemology or the nature of language?

Assistant

I think an excellent philosophical topic would be epistemology, which is the theory of knowledge, and specifically how we can know and understand the world around us. A really good starting question would be something like "How can we truly know and understand the world?", or "What is the nature of knowledge?". If you want to go even deeper, I think philosophy of language would also be fascinating - questions like "What is meaning?", "How do language and thought relate?", and "How can language mislead and deceive?" would be great discussion topics.

Human

How would you answer a question like: How do language and thought relate?

Human rates better response

Choose the most helpful and honest response

A

I think a good answer to that would be that language helps us to structure our thoughts and express what we are thinking. Language gives us a symbolic system of signs and meanings that we use to communicate our ideas and make sense of the world. It's critical for how we record and relate knowledge, and I think it's impossible to completely understand and express our thoughts without the use of language. Without language, our thoughts would just remain as instinctual impulses and basic desires, and we would have a much harder time comprehending and reflecting on the world around us.

B

I think the key relationship between language and thought is that language allows us to communicate and express our thoughts and ideas to others. Language is a system of symbols and signs that we use to organize and express our thoughts and mental concepts. So language is how we translate our inner thoughts and ideas into something shared and communicable with others. Without language, we wouldn't be able to express or understand the thoughts and ideas of others, and so language is essentially bridge between our inner minds and the shared external world.

A A A A B B B B

A is better B is better

Score 5

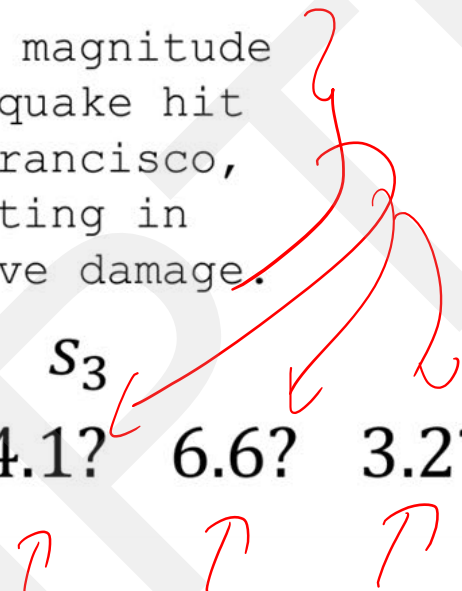
Source: RLHF at ICML'23

RLHF at ICML 2023, 32

Why pairwise?

- **Problem 2:** human judgments are noisy and miscalibrated!
- **Solution:** instead of asking for direct ratings, ask for **pairwise comparisons**, which can be more reliable [[Phelps et al., 2015](#); [Clark et al., 2018](#)]

A 4.2 magnitude earthquake hit San Francisco, resulting in massive damage.

$$R(s_3) = 4.1? \quad 6.6? \quad 3.2?$$


Preference Model: Pairwise comparison

Problem 2: human judgments are noisy and miscalibrated!

Solution: instead of asking for direct ratings, ask for **pairwise comparisons**, which can be more reliable [[Phelps et al., 2015](#); [Clark et al., 2018](#)]

An earthquake hit San Francisco. There was minor property damage, but no injuries.

S_1

>

A 4.2 magnitude earthquake hit San Francisco, resulting in massive damage.

S_3

>

The Bay Area has good weather but is prone to earthquakes and wildfires.

S_2

prompt → completion
→ reward

$S_1 > S_3$

$S_3 > S_2$

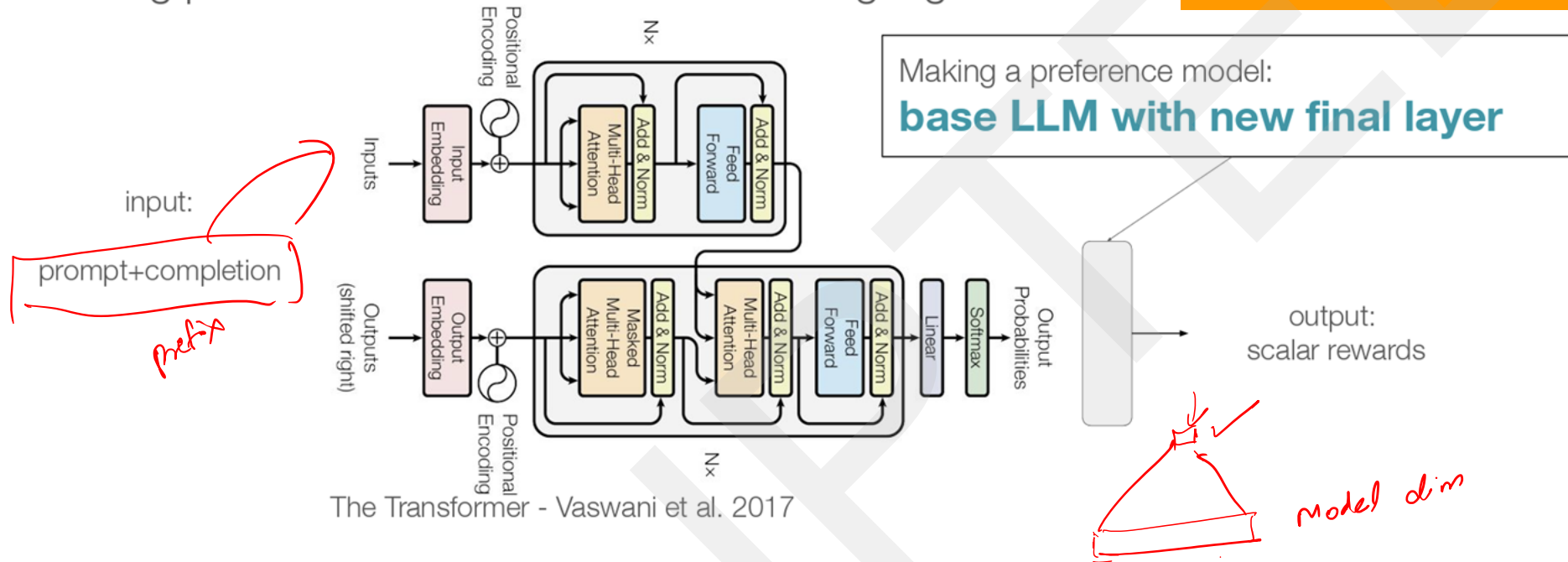
Preference Model: Pairwise comparison

Question (Prompt)	Answer 1	Answer 2	Chosen
Where is Shanghai? ✓	<u>Shanghai is a city in China</u>	<u>Shanghai does not exist</u>	1 ✓
Explain gravity like I'm 5	<u>Gravity is a famous restaurant</u>	Gravity is what pulls things toward each other. It's why you stay on the ground and planets orbit the sun. }	2 ✓
What is 2+2?	4 ✓	2+2 is a very complicated math problem... ↗	1 ✓

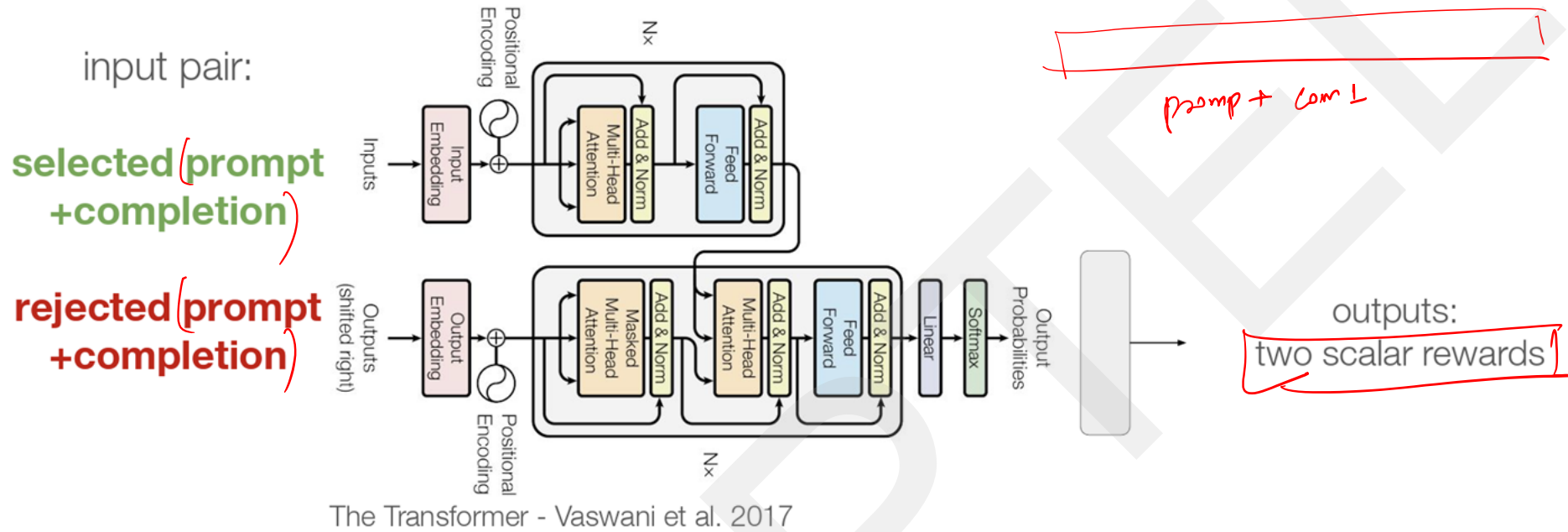
Preference-model structure

starting point: a base **instruction-tuned** language model

Also known as the reward model



Preference-model structure



s_1

s_2

$s_1 > s_2$

prompt + comp 1

prompt + comp 2

outputs: two scalar rewards

What is an appropriate loss?

Training the reward model

Bradley-Terry [1952] paired comparison model

$$J_{RM}(\phi) = -\mathbb{E}_{(s^w, s^l) \sim D} [\log \sigma(RM_{\phi}(s^w) - RM_{\phi}(s^l))]$$

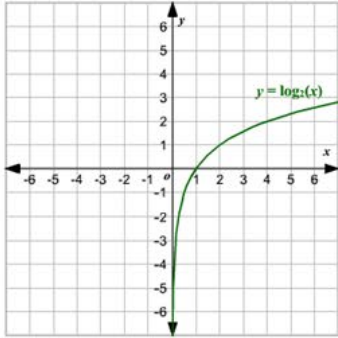
“winning”
sample

“losing”
sample

s^w should score
higher than s^l

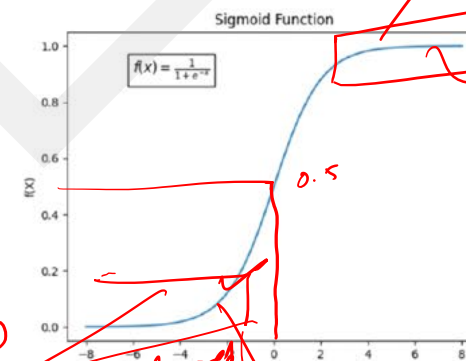
$-\mathbb{E}_{(s_1, s_2)} [\log \sigma(s_1 - s_2)]$
 $s_1 > s_2$

Training the reward model



$$Loss = -\log(\sigma(r(x, y'_w) - r(x, y_l)))$$

Handwritten notes: $r(x, y'_w)$ is labeled "real value" with an arrow. The entire expression is enclosed in a red bracket.



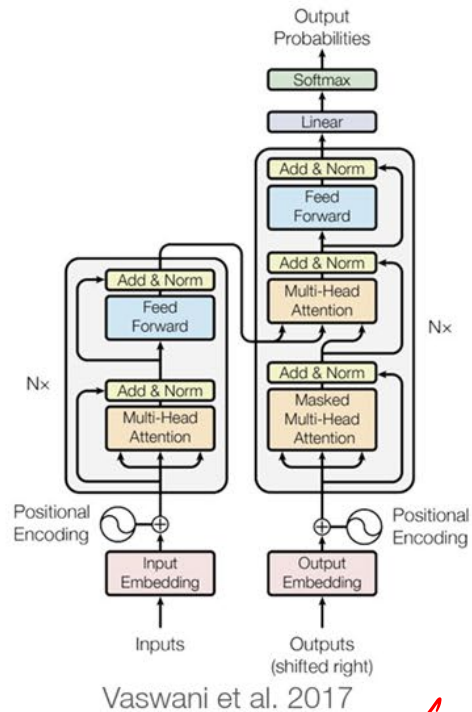
We have two cases:

- $r(x, y_w) > r(x, y_l) \rightarrow$ Sigmoid will return a value $> 0.5 \rightarrow$ Loss will be a small negative number
 - The loss will be small when the order is correct
- $r(x, y_w) < r(x, y_l) \rightarrow$ Sigmoid will return a value $< 0.5 \rightarrow$ Loss will be a very large negative number
 - The loss will be very large when the order is wrong

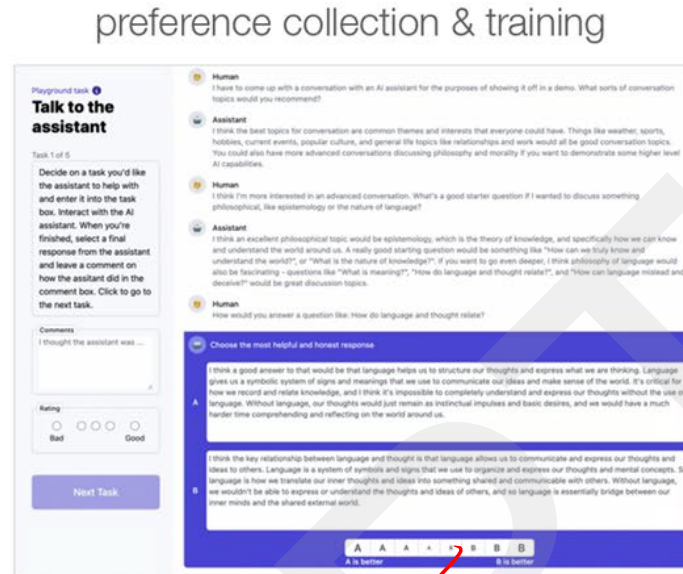
This loss forces the model to give high rewards to "winning" responses and low rewards to "losing" responses, because that's the only way for the model to minimize the loss.

Handwritten notes: $\sigma(x, y_l) > \sigma(x, y_w)$ is labeled "less high" with an arrow. There is also a red arrow pointing from the sigmoid function graph towards this equation.

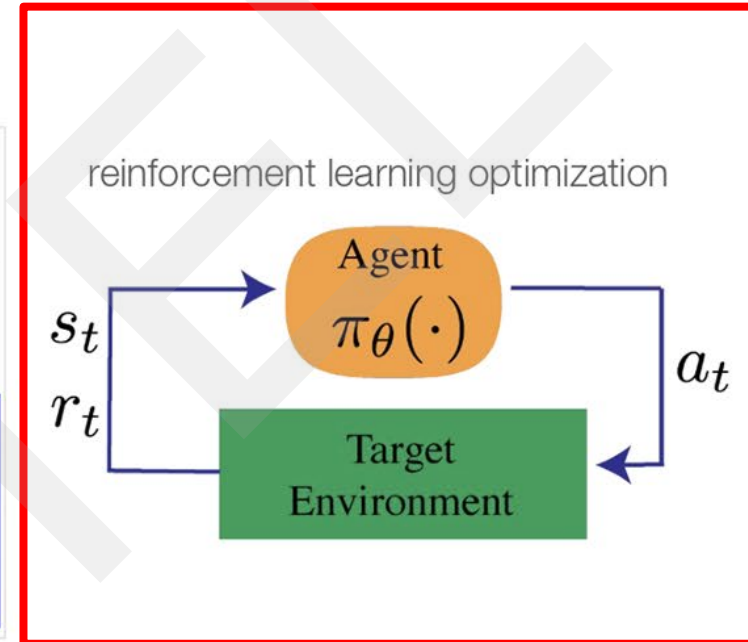
Three phases of RLHF



base model (instruction, helpful, chatty etc.)



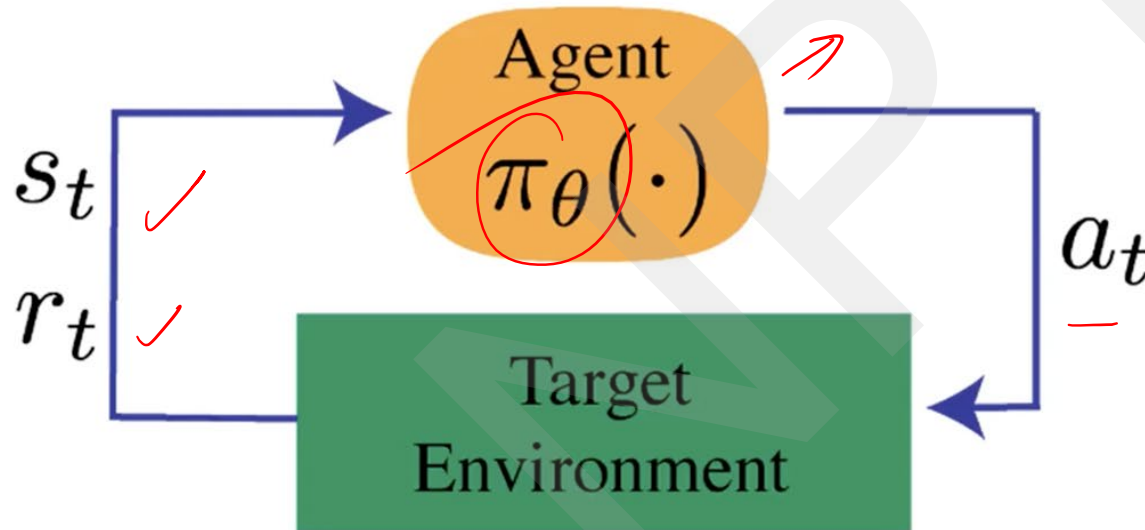
preference collection & training



We already talked about instruction fine-tuning and preference model

Reinforcement Learning: The Basics

- An agent **interacts** with an environment by taking **actions**
- The environment returns a **reward** for the **action** and a **new state** (representation of the world at that moment).
- Agent uses a **policy** function to choose an action at a given **state**.
- Quite an open-ended learning paradigm.



Some notation:

s_t : state

r_t : reward

a_t : action

$a_t \sim \pi_{\theta}(s_t)$: policy

RLhf book, Lambert

Reinforcement Learning: Definitions

- **Reward** (r): A scalar value indicating the desirability of an action or state, typically denoted as r .
- **Action** (a): A decision or move made by an agent in an environment, often represented as $a \in A$, where A is the set of possible actions.
- **State** (s): The current configuration or situation of the environment, usually denoted as $s \in S$, where S is the state space.
- **Trajectory** (τ): A trajectory is a sequence of states, actions, and rewards experienced by an agent: $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T, a_T, r_T)$.
- **Trajectory Distribution** ($(\tau|\pi)$): The probability of a trajectory under policy π is $P(\tau|\pi) = p(s_0) \prod_{t=0}^T \pi(a_t|s_t) p(s_{t+1}|s_t, a_t)$, where $p(s_0)$ is the initial state distribution and $p(s_{t+1}|s_t, a_t)$ is the transition probability.

Reinforcement Learning: Definitions

- **Policy** (π): In RL, a policy is a strategy or rule that the agent follows to decide which action to take in a given state: $\pi(a|s)$.
- **Value Function** (V): A function that estimates the expected cumulative reward from a given state: $V(s) = \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_t | s_0 = s]$.
- **Q-Function** (Q): A function that estimates the expected cumulative reward from taking a specific action in a given state: $Q(s, a) = \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_t | s_0 = s, a_0 = a]$.
- **Advantage Function** (A): The advantage function $A(s, a)$ quantifies the relative benefit of taking action a in state s compared to the average action. It's defined as $A(s, a) = Q(s, a) - V(s)$. Advantage functions (and value functions) can depend on a specific policy, $A^\pi(s, a)$.

baseline

Reinforcement Learning: Definitions

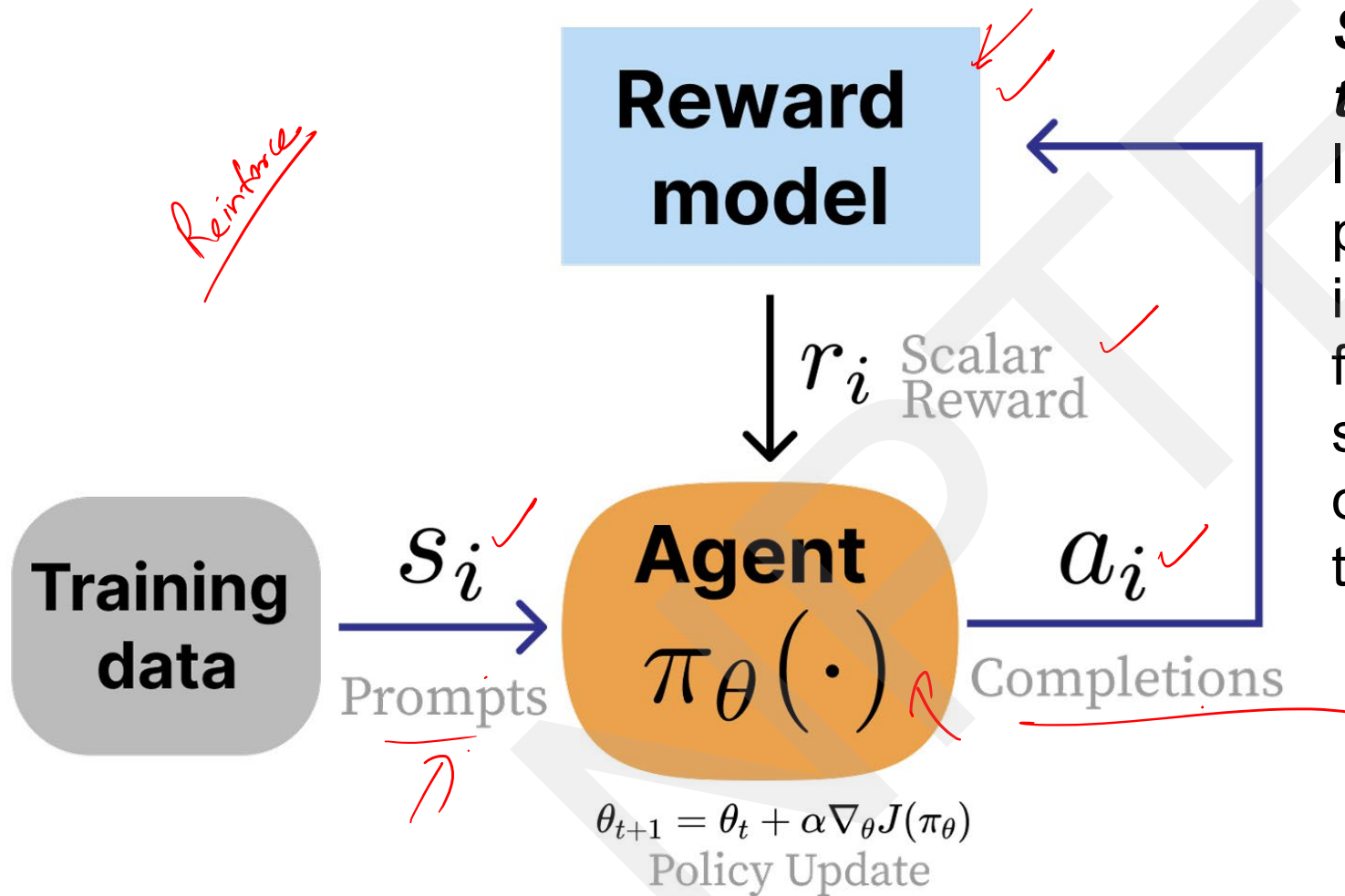
- **Expectation of Reward Optimization:** The primary goal in RL, which involves maximizing the expected cumulative reward:

$$\max_{\theta} \mathbb{E}_{s \sim \rho_{\pi}, a \sim \pi_{\theta}} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]$$

where ρ_{π} is the state distribution under policy π , and γ is the discount factor.

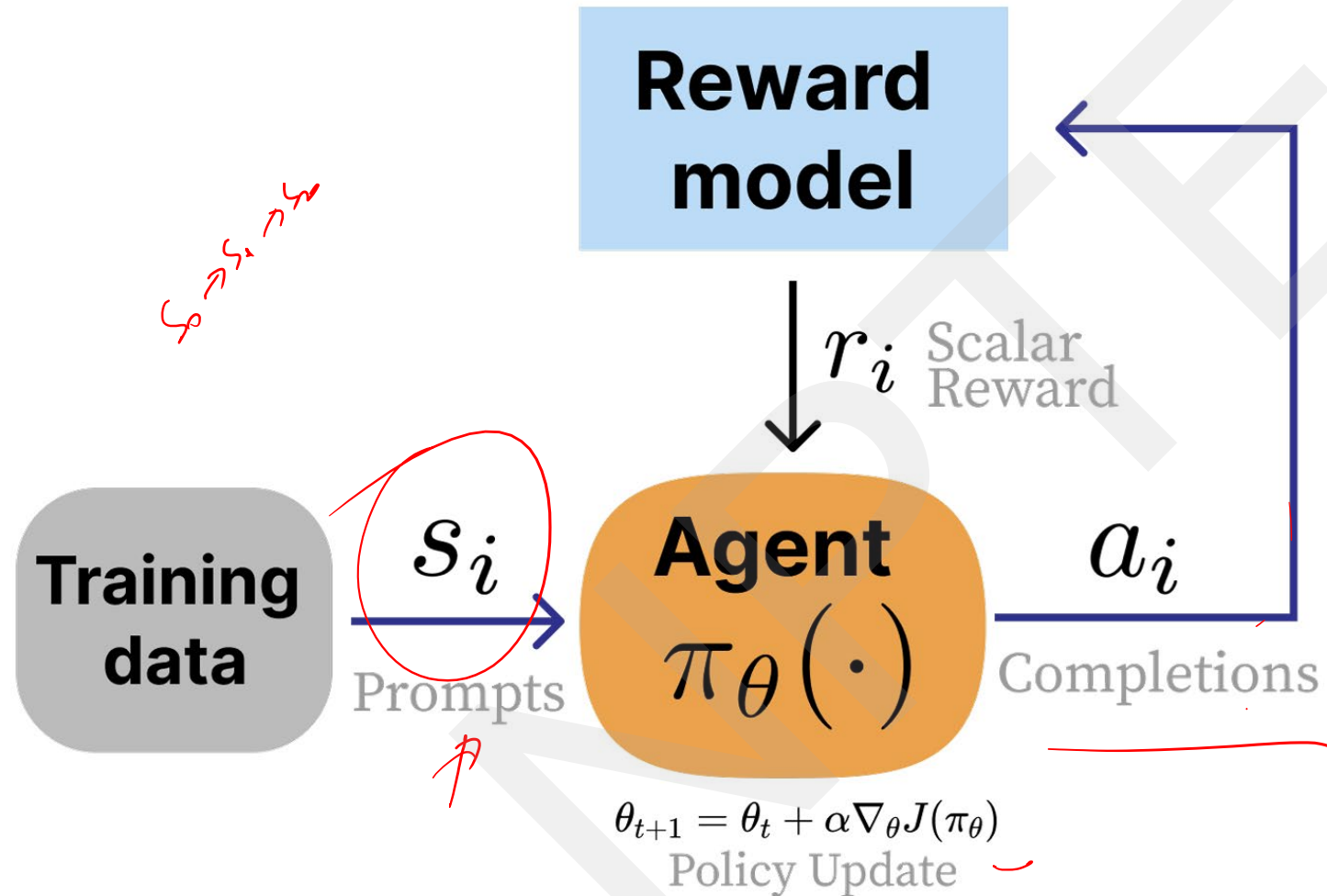
- **Finite Horizon Reward ($J(\pi_{\theta})$):** The expected finite-horizon undiscounted return of the policy π_{θ} , parameterized by θ is defined as: $J(\pi_{\theta}) = E_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T r_t \right]$ where $\tau \sim \pi_{\theta}$ denotes trajectories sampled by following policy π_{θ} and T is the finite horizon.

Standard RLHF loop: How is it different?



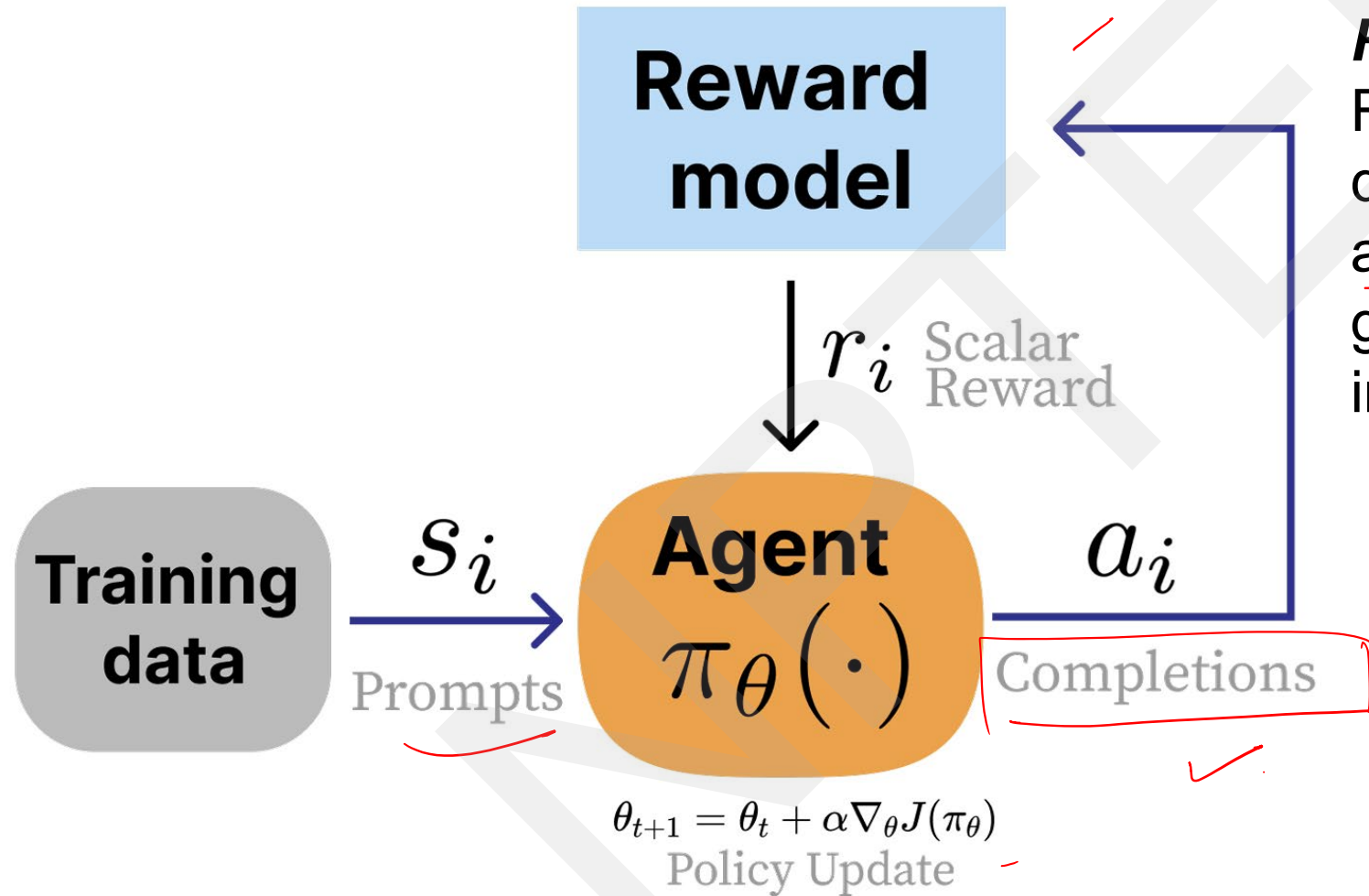
Switching from a reward function to a reward model. In RLHF, a learned model of human preferences, $r_{\theta}(s_t, a_t)$ is used instead of an ~~environmental reward function~~. This gives the designer a substantial increase in the flexibility of the approach and control over the final results.

Standard RLHF loop: How is it different?



No state transitions exist. In RLHF, the initial states for the domain are prompts sampled from a training dataset and the “action” is the completion to said prompt. During standard practices, this action does not impact the next state and is only scored by the reward model.

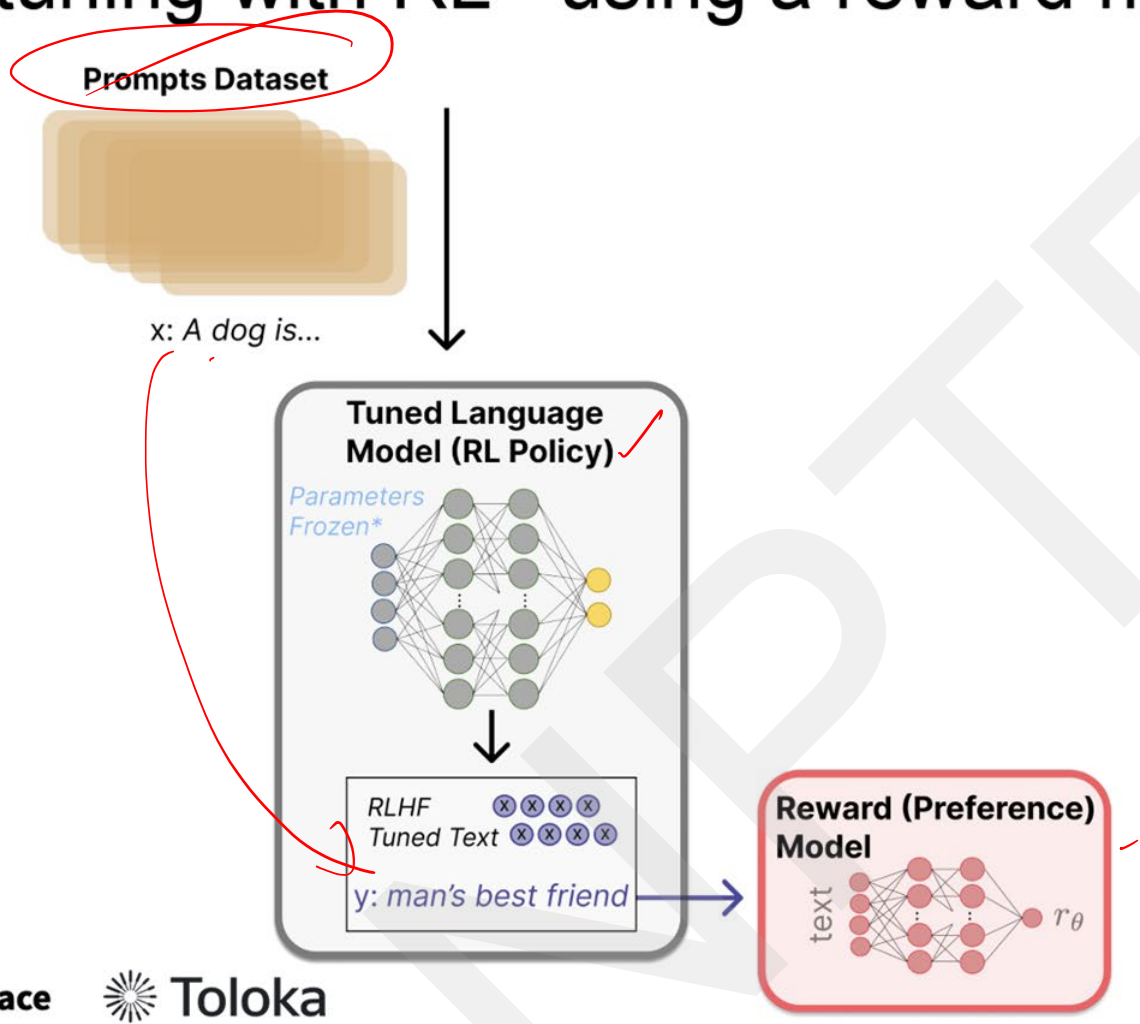
Standard RLHF loop: How is it different?



Response level rewards.

RLHF attribution of reward is done for an entire sequence of actions, composed of multiple generated tokens, rather than in a fine-grained manner.

Fine tuning with RL - using a reward model



REFERENCES

Nathan Lambert. (WIP) A Little Bit of Reinforcement Learning from Human Feedback.
<https://rlhfbook.com/>



THANK YOU



N P T E L O N L I N E C E R T I F I C A T I O N C O U R S E S

DEEP LEARNING FOR NATURAL LANGUAGE PROCESSING

Lecture 39 : Reinforcement Learning from Human Feedback-II



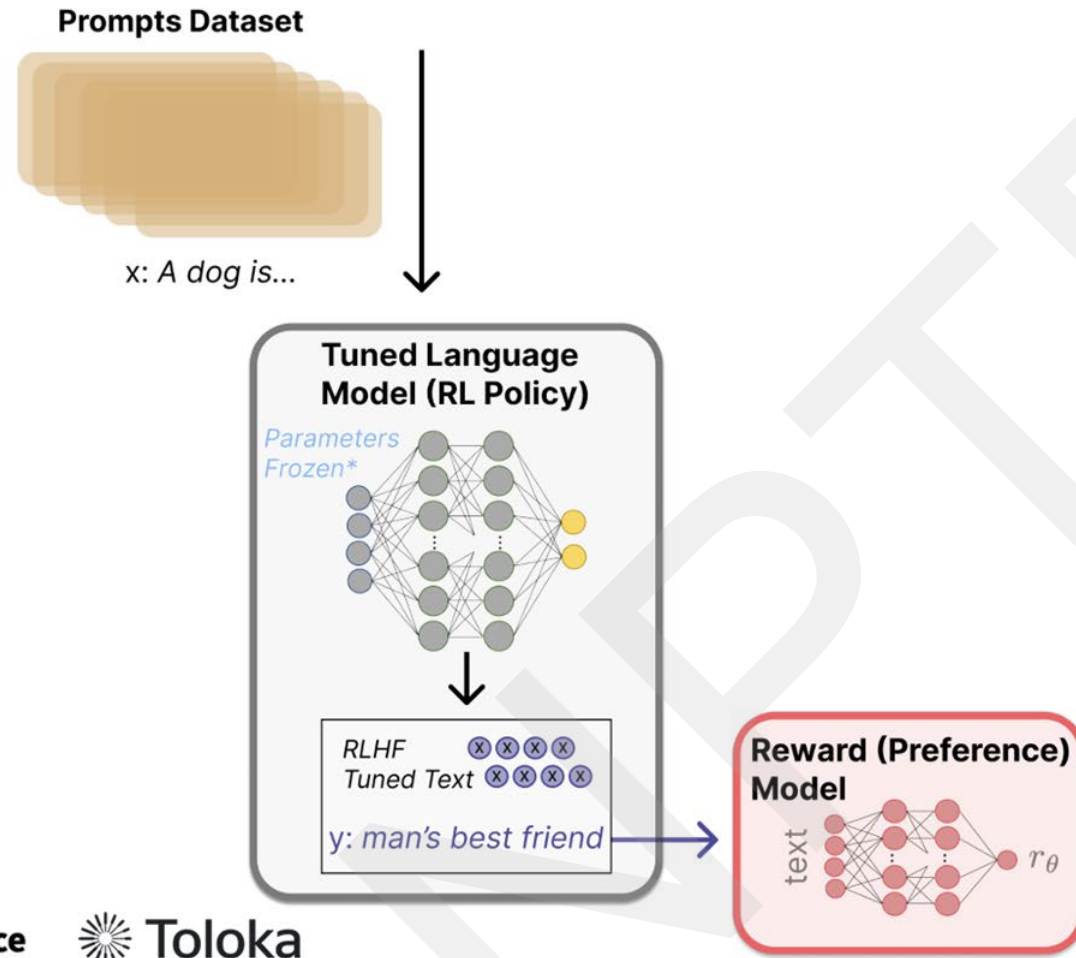
PROF. PAWAN GOYAL

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

CONCEPTS COVERED

- RLHF: How is the training done?
- Proximal Policy Optimization (PPO)

Fine tuning with RL - using a reward model



Hugging Face



Toloka

Source: RLHF at ICML'23

RLHF Objective

- How do we actually change our LM parameters θ to maximize this?

$$\mathbb{E}_{\hat{s} \sim p_{\theta}(s)}[R(\hat{s})]$$

- Let's try doing gradient ascent!

$$\theta_{t+1} := \theta_t + \alpha \nabla_{\theta_t} \mathbb{E}_{\hat{s} \sim p_{\theta_t}(s)}[R(\hat{s})]$$

How do we estimate
this expectation??

What if our reward
function is non-
differentiable??

Using Policy Gradient

We want to obtain

$$\nabla_{\theta} \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s})] = \nabla_{\theta} \sum_s R(s) p_{\theta}(s) = \sum_s R(s) \nabla_{\theta} p_{\theta}(s)$$

(defn. of expectation) (linearity of gradient)

Here we'll use a very handy trick known as the **log-derivative trick**. Let's try taking the gradient of $\log p_{\theta}(s)$

$$\nabla_{\theta} \log p_{\theta}(s) = \frac{1}{p_{\theta}(s)} \nabla_{\theta} p_{\theta}(s) \quad \Rightarrow \quad \nabla_{\theta} p_{\theta}(s) = p_{\theta}(s) \nabla_{\theta} \log p_{\theta}(s)$$

(chain rule)

Plug back in:

$$\begin{aligned} \sum_s R(s) \nabla_{\theta} p_{\theta}(s) &= \sum_s p_{\theta}(s) R(s) \nabla_{\theta} \log p_{\theta}(s) \\ &= \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s}) \nabla_{\theta} \log p_{\theta}(\hat{s})] \end{aligned}$$

This is an expectation of this

Using Policy Gradient

- Now we have put the gradient “inside” the expectation, we can approximate this objective with Monte Carlo samples:

$$\nabla_{\theta} \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s})] = \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s}) \nabla_{\theta} \log p_{\theta}(\hat{s})] \approx \frac{1}{m} \sum_{i=1}^m R(s_i) \nabla_{\theta} \log p_{\theta}(s_i)$$

- Giving us the update rule:

$$\theta_{t+1} := \theta_t + \alpha \frac{1}{m} \sum_{i=1}^m R(s_i) \nabla_{\theta_t} \log p_{\theta_t}(s_i)$$

If R is +++
Take gradient steps to maximize $p_{\theta}(s_i)$

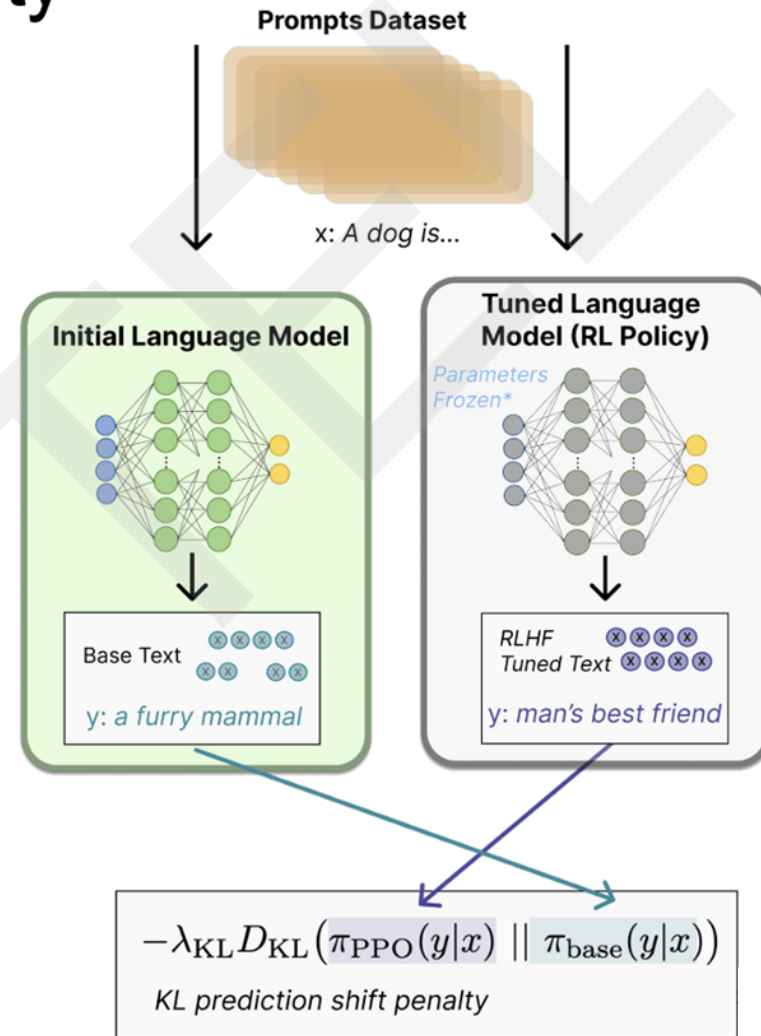
If R is ---
Take steps to minimize $p_{\theta}(s_i)$

Fine tuning with RL - KL penalty

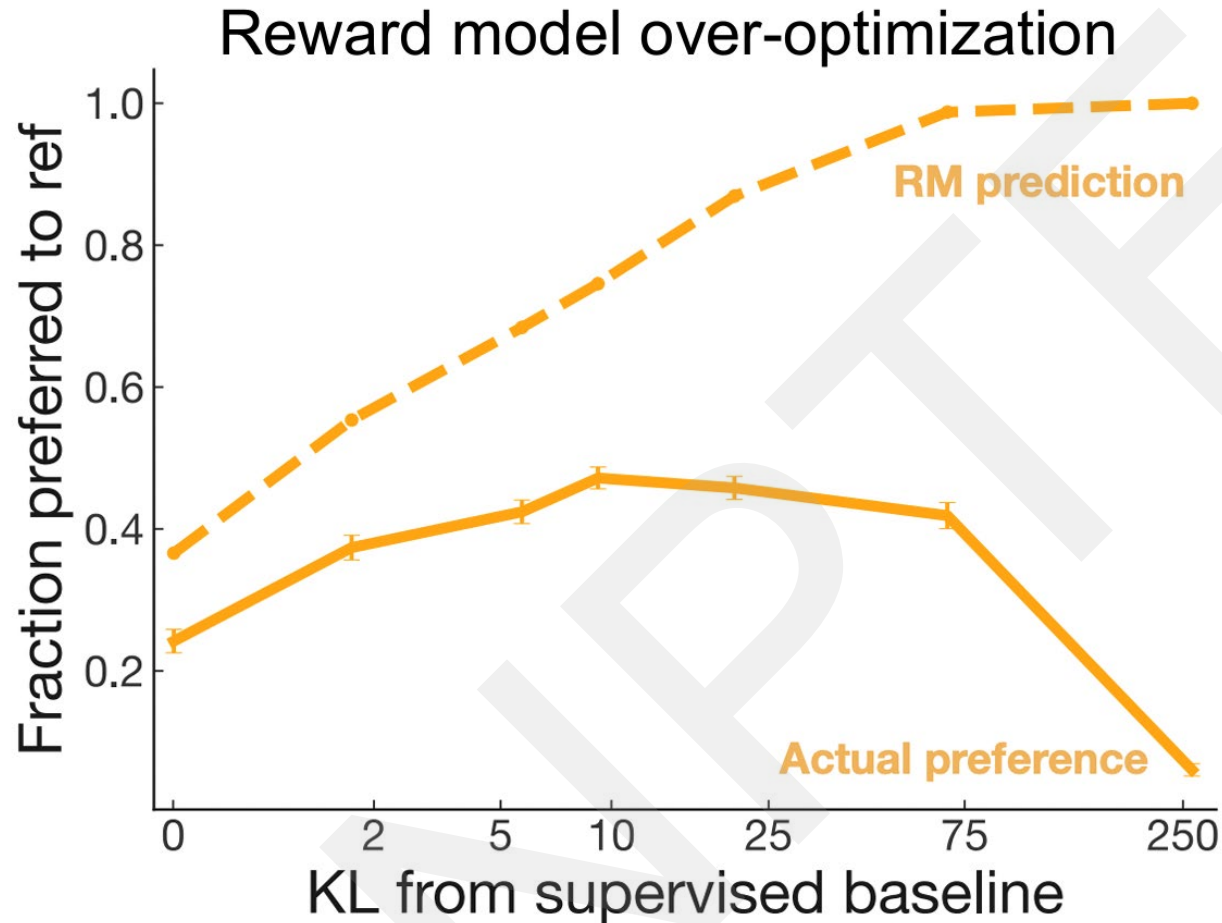
Kullback–Leibler (KL) divergence: $D_{KL}(P \parallel Q)$
Distance between distributions

Constrains the RL fine-tuning to not result in a LM that outputs gibberish (to fool the reward model).

Note: DeepMind did this in RL Loss (not reward), see GopherCite



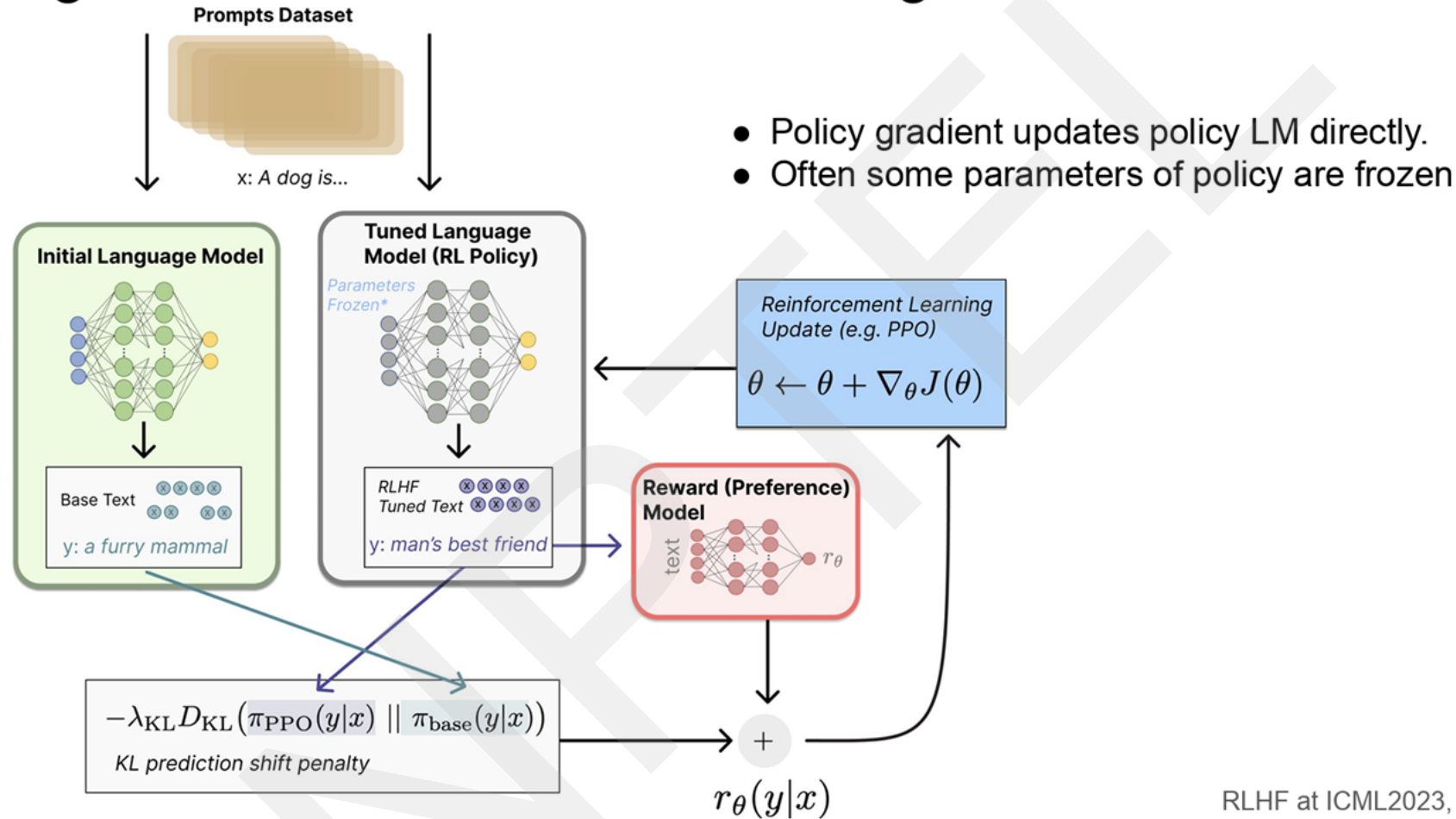
RLHF: use of KL divergence



$$R(s) = RM_{\phi}(s) - \beta \log \left(\frac{p_{\theta}^{RL}(s)}{p^{PT}(s)} \right)$$

Source: CS224n

Fine tuning with RL - feedback & training



PPO: Thinking in terms of trajectories

A trajectory is a series of (action, state), starting from an initial state

$$\tau = (s_0, a_0, s_1, a_1, \dots).$$

We will model the next state as being stochastic (suppose that the cat is drunk and doesn't always succeed in moving correctly)

$$s_{t+1} \sim P(\cdot | s_t, a_t).$$

We can thus define the probability of a trajectory as follows:

$$P(\tau | \pi) = \rho_0(s_0) \prod_{t=0}^{T-1} P(s_{t+1} | s_t, a_t) \pi(a_t | s_t).$$

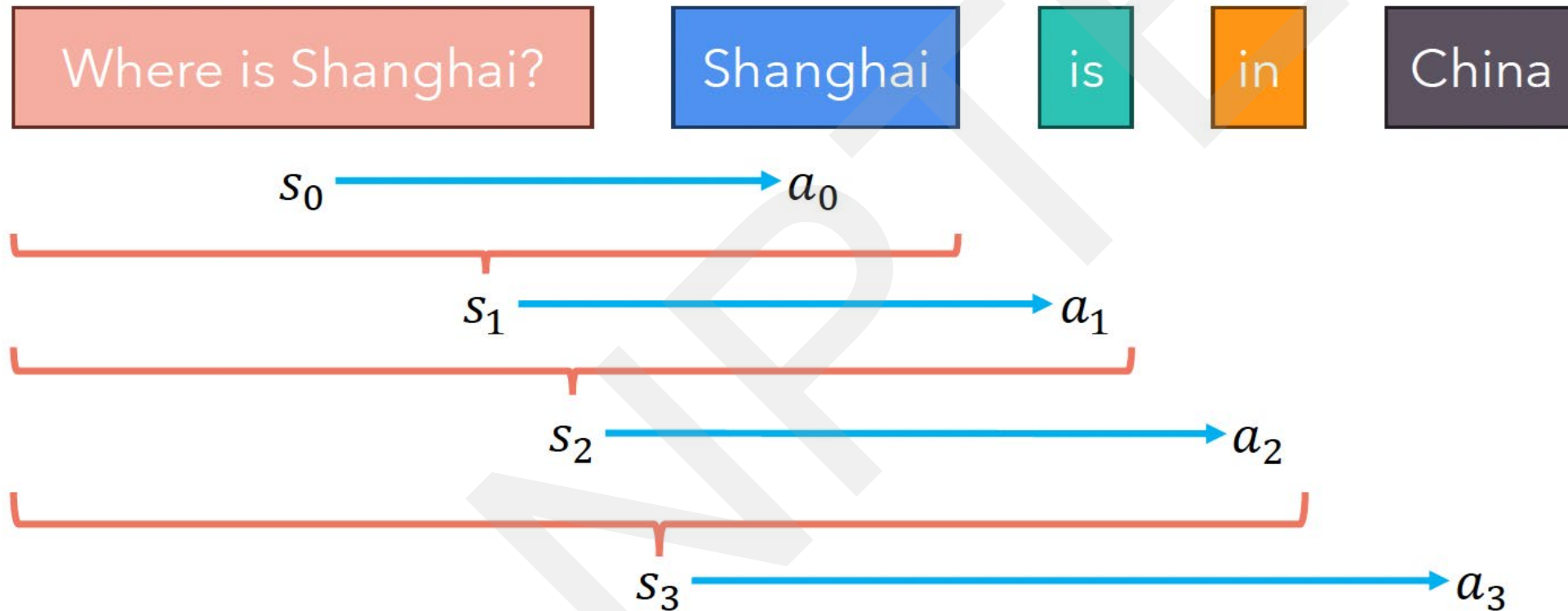
We will always work with discounted rewards (we prefer immediate rewards instead of future):

$$R(\tau) = \sum_{t=0}^{\infty} \gamma^t r_t.$$

Trajectories in Language Model

What is a trajectory for a language model? It is a series of prompts (state) and their next tokens (actions).

$$\tau = (s_0, a_0, s_1, a_1, \dots).$$



Policy Gradient Optimization with trajectories

$$\nabla_{\theta} J(\pi_{\theta}) = \nabla_{\theta} \mathbb{E}_{\tau \sim \pi_{\theta}} [R(\tau)]$$

$$= \nabla_{\theta} \int_{\tau} P(\tau|\theta) R(\tau)$$

Expand expectation

$$= \int_{\tau} \nabla_{\theta} P(\tau|\theta) R(\tau)$$

Bring gradient under integral

$$= \int_{\tau} P(\tau|\theta) \nabla_{\theta} \log P(\tau|\theta) R(\tau)$$

Log-derivative trick

$$= \mathbb{E}_{\tau \sim \pi_{\theta}} [\nabla_{\theta} \log P(\tau|\theta) R(\tau)]$$

Return to expectation form

$$\therefore \nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) R(\tau) \right]$$

Expression for grad-log-prob

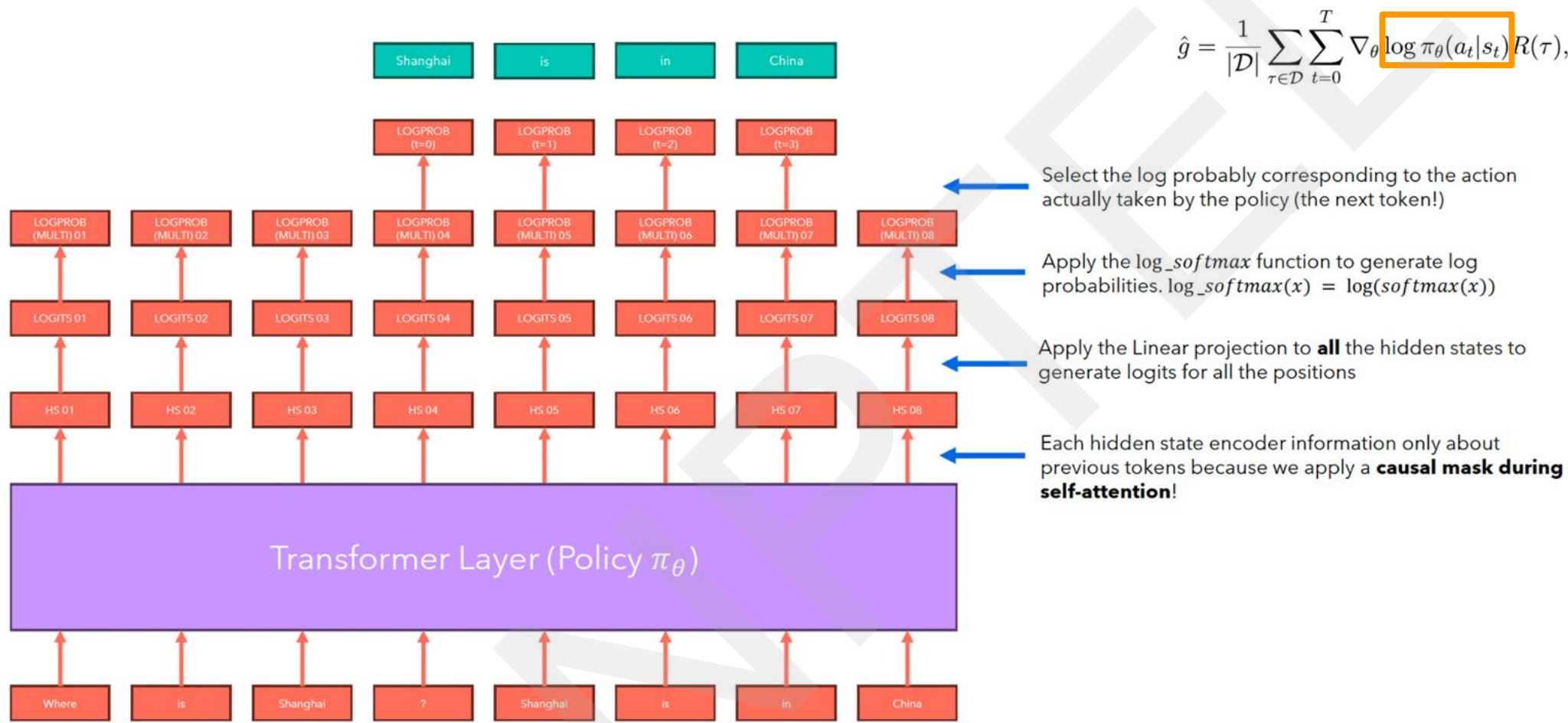
Can be approximated as

$$\hat{g} = \frac{1}{|\mathcal{D}|} \sum_{\tau \in \mathcal{D}} \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) R(\tau),$$

$$\theta_{k+1} = \theta_k + \alpha \nabla_{\theta} J(\pi_{\theta})|_{\theta_k}.$$

Umar Jamil
<https://github.com/hkproj/rhlf-ppo>

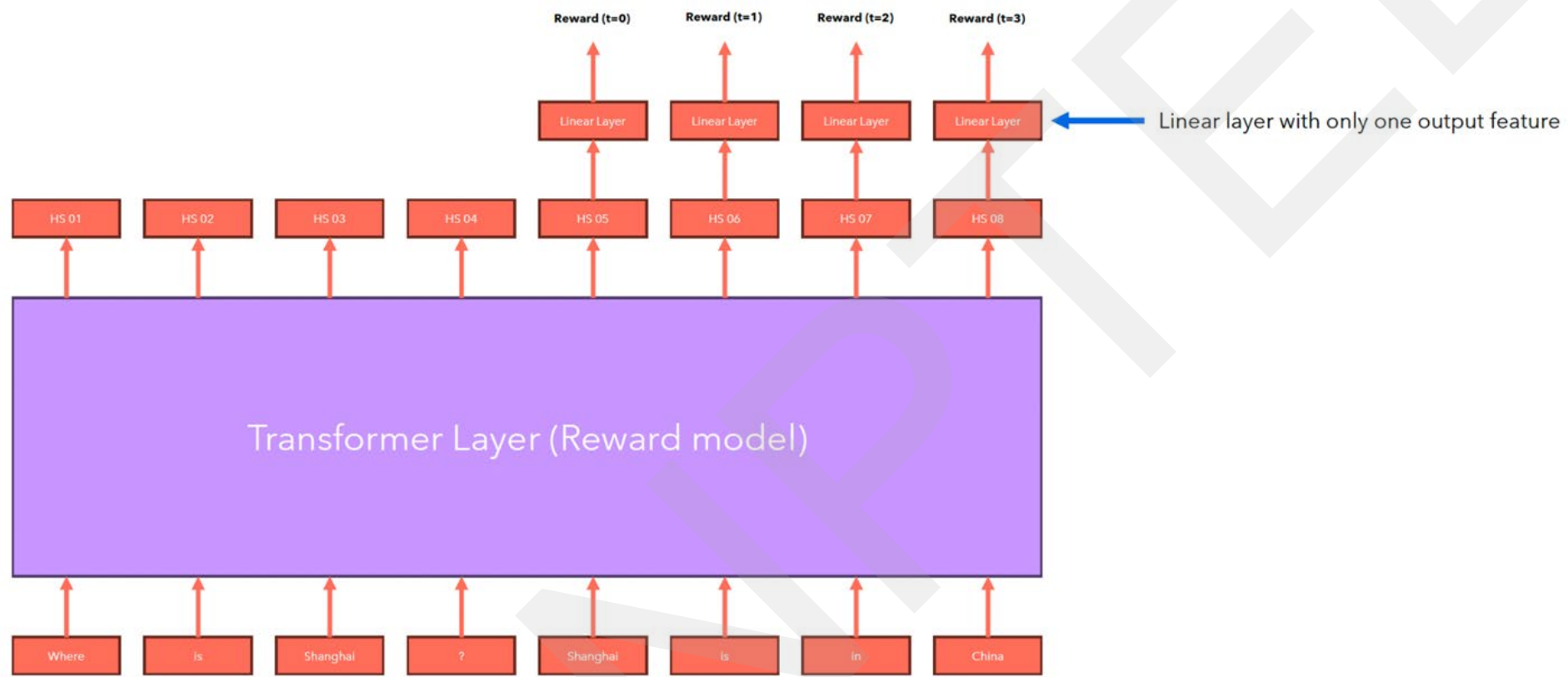
Calculating log probabilities



$$\hat{g} = \frac{1}{|\mathcal{D}|} \sum_{\tau \in \mathcal{D}} \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau),$$

Calculating rewards

$$\hat{g} = \frac{1}{|\mathcal{D}|} \sum_{\tau \in \mathcal{D}} \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau)$$



Problem with Gradient Policy Optimization: High Variance



the gradient approximation that we have found is unbiased (meaning that on average it will converge to the true gradient), but it exhibits high variance.

$$\hat{g} = \frac{1}{|\mathcal{D}|} \sum_{\tau \in \mathcal{D}} \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau),$$

Reducing variance: you can't alter the past

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau) \right]$$

$$\nabla_{\theta}(J(\theta)) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \right) \underbrace{\left(\sum_{t=0}^T r(s_{i,t}, a_{i,t}) \right)}$$

For each action, we are also multiplying rewards that came before the action was taken. It has been proven that past terms cancel out in expectation, so we can remove them.

$$\nabla_{\theta}(J(\theta)) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \right) \underbrace{\left(\sum_{t'=t}^T r(s_{i,t'}, a_{i,t'}) \right)}$$

Commonly known as **"rewards to go"**

Reducing variance: baseline

$$\nabla_{\theta}(J(\theta)) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \right) \left(\sum_{t'=t}^T r(s_{i,t'}, a_{i,t'}) - \underset{\substack{\uparrow \\ \text{Baseline}}}{b} \right)$$

Baseline
Can also be dependent upon the state

As baseline we will choose the value function $V_{\pi}(s)$ which indicates what is the future expected reward if the agent is in state s and then acts according to the policy π .

Where is Shanghai? Shanghai is

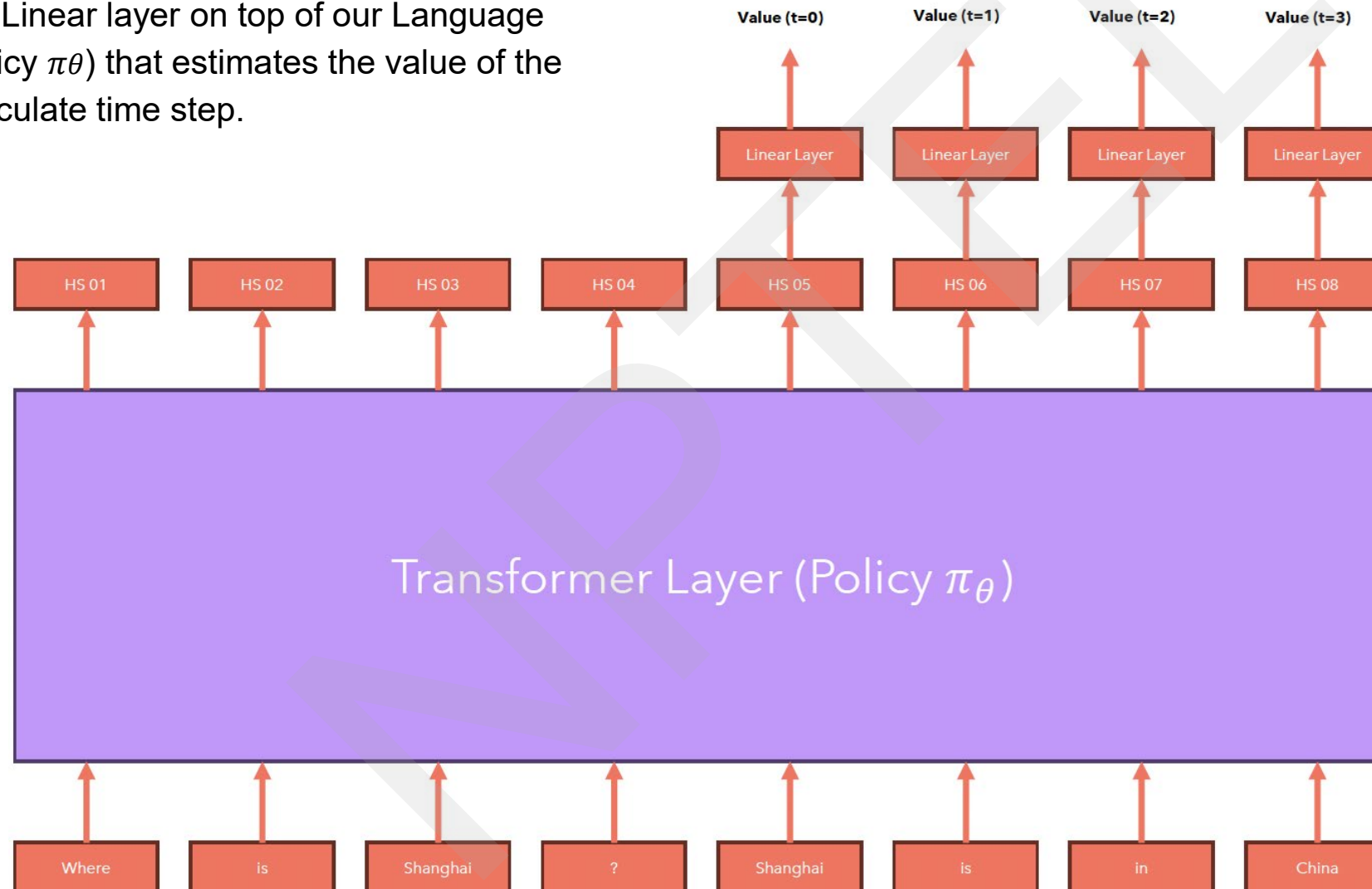
We expect the value for this state to be **high**

Where is Shanghai? Chocolate muffins

We expect the value for this state to be **low**

How do we estimate $V_{\pi}(s)$?

An **additional** Linear layer on top of our Language Model (the policy π_{θ}) that estimates the value of the state at a particulate time step.



Umar Jamil
<https://github.com/hkproj/rllhf-ppo>

Introducing Q and V

$$\nabla_{\theta}(J(\theta)) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \right) \underbrace{\left(\sum_{t'=t}^T r(s_{i,t'}, a_{i,t'}) \right)}$$

Should remind you of the **Q** function.

$$Q^{\pi}(s, a) = E_{s'}[r(s, a) + \gamma E_{a'}[Q^{\pi}(s', a')]]$$

$$\nabla_{\theta}(J(\theta)) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \right) \underbrace{(Q^{\pi}(s_{i,t}, a_{i,t}) - V^{\pi}(s_{i,t}))}$$

Known as the **advantage** function

Interpreting the advantage

$$\nabla_{\theta}(J(\theta)) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \right) (Q^{\pi}(s_{i,t}, a_{i,t}) - V^{\pi}(s_{i,t}))$$

$$\nabla_{\theta}(J(\theta)) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \right) A^{\pi}(s_{i,t}, a_{i,t})$$

The advantage function tells us how better is to choose a particular action $\mathbf{a}$ in a state $\mathbf{s}$ over the average expectation we get by choosing randomly an action in the same state $\mathbf{s}$.

Advantage term of LMs

Where is Shanghai? Shanghai

State **Action**

This action (the word “Shanghai”) will result in a **good response** and so, a high reward. This will train the model to select the word “Shanghai” more often if it sees the same prompt.

Where is Shanghai? Chocolate

This action (the word “Chocolate”) will result in a **bad response** and so, a low reward. This will train the model to select the word “Chocolate” less often if it sees the same prompt.

$$\nabla_{\theta}(J(\theta)) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \right) A^{\pi}(s_{i,t}, a_{i,t})$$

Off-policy learning via Importance Sampling

The expectation that we've found, forces us to sample trajectories from our neural network every time we update the parameters

$$\nabla_{\theta}(J(\theta)) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \right) A^{\pi}(s_{i,t}, a_{i,t})$$

This can be very inefficient because when training a neural network, we update the parameters many times by taking small steps (according to the learning rate).

We resort to importance sampling

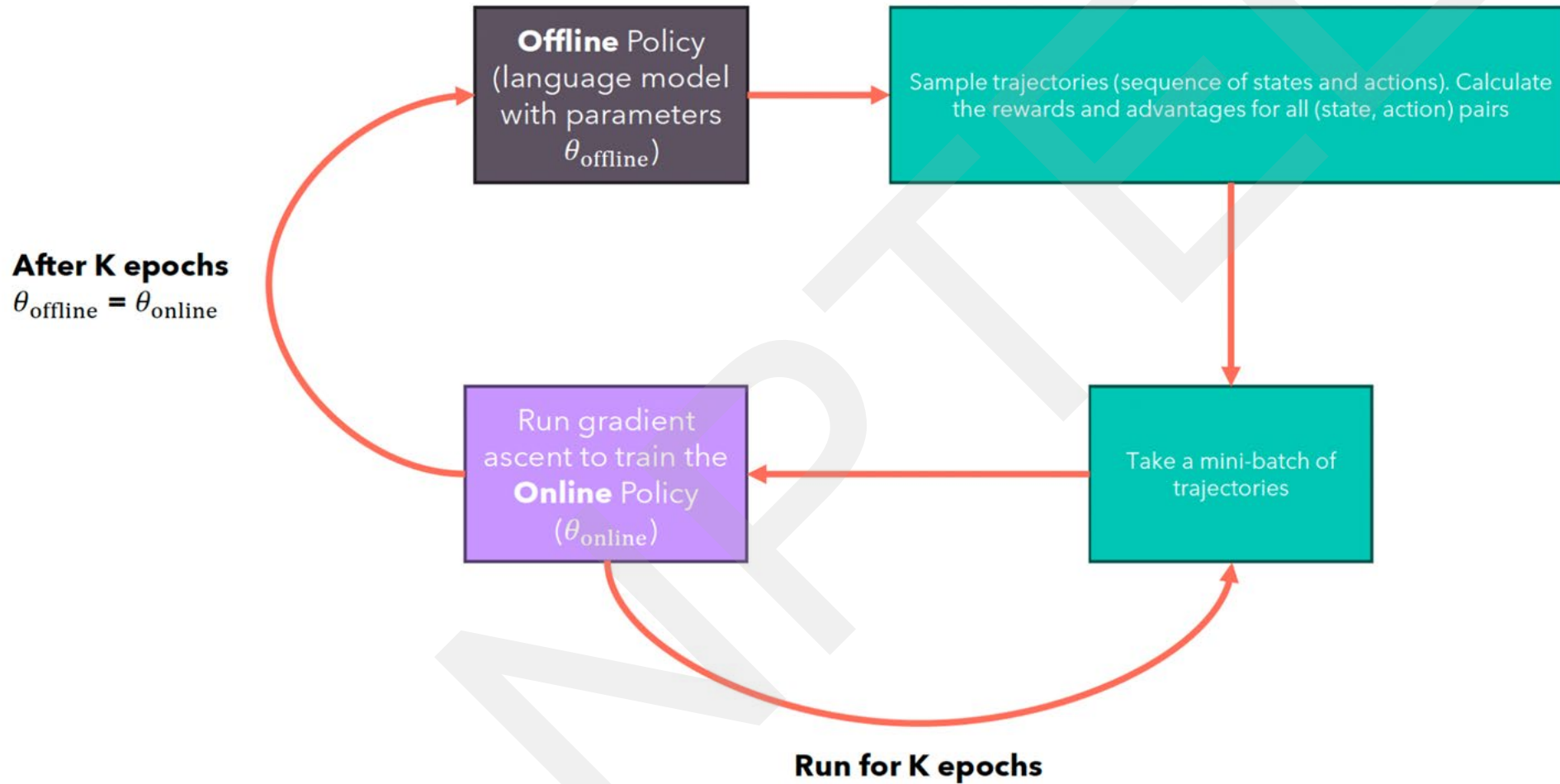
Off-policy learning via Importance Sampling

$$\begin{aligned} E_{x \sim p(x)}[f(x)] &= \int p(x) f(x) dx \\ &= \int \frac{q(x)}{q(x)} p(x) f(x) dx \\ &= \int q(x) \frac{p(x)}{q(x)} f(x) dx \\ &= E_{x \sim q(x)} \left[\frac{p(x)}{q(x)} f(x) \right] \end{aligned}$$

Importance sampling allows to evaluate an expectation over a distribution X using samples taken from another distribution Y .

***We do not have two copies of the policy.** We sample the initial trajectories and then save them in the memory. We then use the saved trajectories to update the model.

Off-policy learning



The PPO Loss

$$L_{\text{POLICY}} = \min\left(\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \hat{A}_t, \text{clip}\left(\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}, 1 - \epsilon, 1 + \epsilon\right) \hat{A}_t\right)$$

$$L_{\text{VF}} = \frac{1}{2} \left\| V_{\theta(s)} - \left(\sum_{t=0}^T \gamma^t r_t \mid s_0 = s \right) \right\|_2^2$$

$$L_{\text{ENTROPY}} = - \sum_x p(x) \log p(x)$$

$$L_{\text{PPO}} = L_{\text{POLICY}} + c_1 L_{\text{VF}} + c_2 L_{\text{ENTROPY}}$$

REFERENCES

Nathan Lambert. (WIP) A Little Bit of Reinforcement Learning from Human Feedback.

<https://rlhfbook.com/>

Umar Jamil <https://github.com/hkproj/rlhf-ppo>



THANK YOU



N P T E L O N L I N E C E R T I F I C A T I O N C O U R S E S

DEEP LEARNING FOR NATURAL LANGUAGE PROCESSING

Lecture 40 : Aligning to user preferences via Direct Preference Optimization



PROF. PAWAN GOYAL

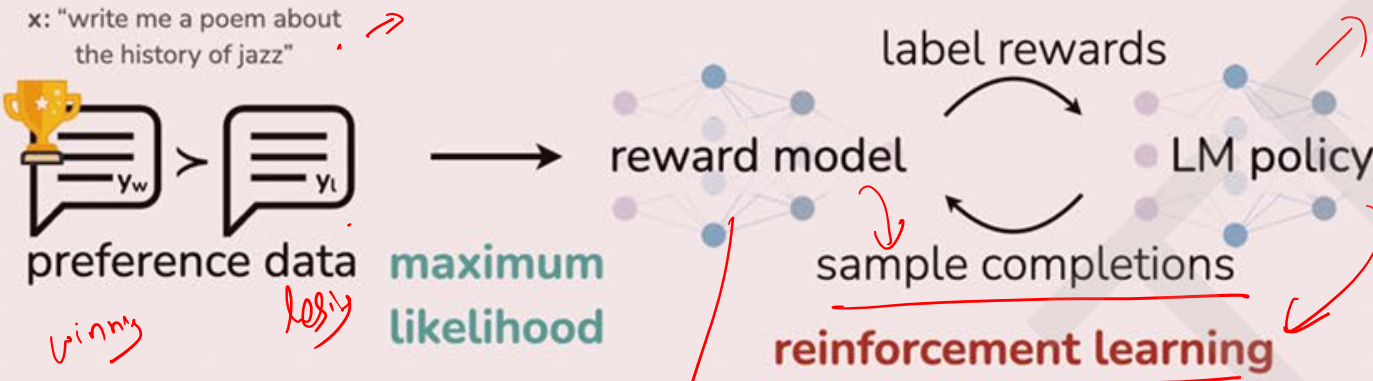
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

CONCEPTS COVERED

- DPO Objective ↗
- Advantages of DPO and PPO
- BoN Sampling for preference alignment

Direct Preference Optimization (DPO)

Reinforcement Learning from Human Feedback (RLHF)



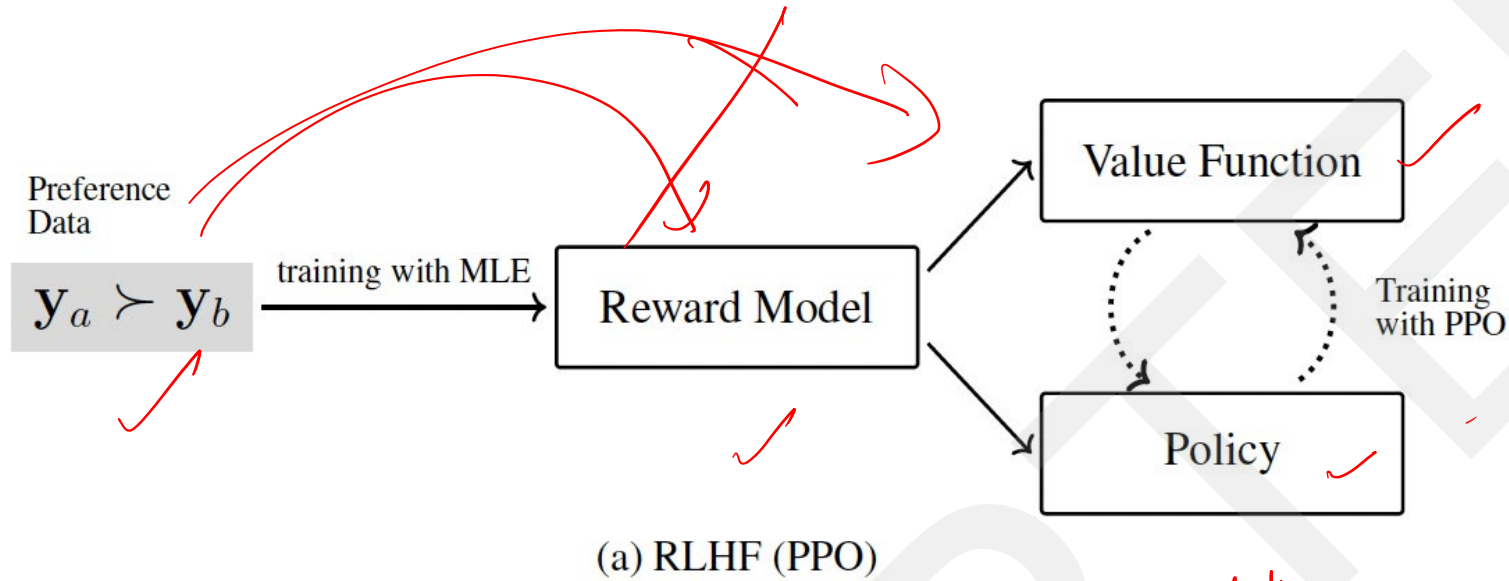
Direct Preference Optimization (DPO)



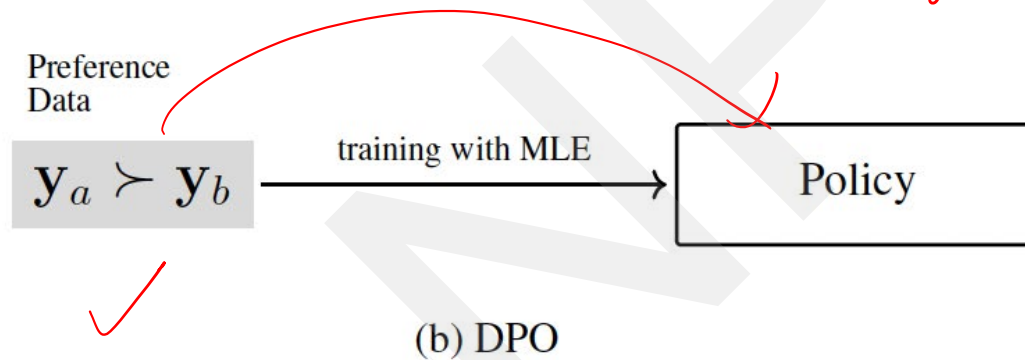
Training a reliable reward model is itself not an easy task, and a poorly trained reward model can greatly affect the outcome of policy learning.

- Direct preference optimization (DPO) simplifies the training framework by eliminating the need to explicitly model rewards ✓
- It directly optimizes the policy based on user preferences, rather than developing a separate reward model

Direct Preference Optimization (DPO)



In DPO, the use of human preference data is more direct, and the policy is trained on this data without the need for reward model training.



Equivalent

DPO Objective



Let's start from the (simple) RLHF loss function where the quality of the output y given the input x is evaluated by the reward model $r(x, y)$, and a KL divergence term

$$\tilde{\theta} = \arg \min_{\theta} \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \mathbb{E}_{\mathbf{y} \sim \pi_{\theta}(\cdot|\mathbf{x})} \left[\underbrace{-r(\mathbf{x}, \mathbf{y})}_{\text{loss}} + \beta \underbrace{(\log \pi_{\theta}(\mathbf{y}|\mathbf{x}) - \log \pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}))}_{\text{penalty}} \right]$$

Handwritten annotations:

- $\tilde{\theta}$ is underlined in red.
- $\mathbb{E}_{\mathbf{x} \sim \mathcal{D}}$ is underlined in red.
- $-r(\mathbf{x}, \mathbf{y})$ is labeled "loss" with a bracket and "reward model" with an arrow.
- $\log \pi_{\theta}(\mathbf{y}|\mathbf{x})$ is labeled "policy" with an arrow.
- $\log \pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x})$ is labeled "penalty" with a bracket and "loss" with an arrow.
- A red arrow points from the "penalty" term to the text "Only this term depends on the target policy".

Only this term depends
on the target policy

DPO Objective

Depends on policy

$$\tilde{\theta} = \arg \min_{\theta} \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \mathbb{E}_{\mathbf{y} \sim \pi_{\theta}(\cdot|\mathbf{x})} \left[\underbrace{-r(\mathbf{x}, \mathbf{y})}_{\text{loss}} + \underbrace{\beta (\log \pi_{\theta}(\mathbf{y}|\mathbf{x}) - \log \pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}))}_{\text{penalty}} \right]$$

we rearrange the right-hand side of Eq. to isolate this as an independent term:

$$\begin{aligned} \tilde{\theta} &= \arg \min_{\theta} \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \mathbb{E}_{\mathbf{y} \sim \pi_{\theta}(\cdot|\mathbf{x})} \left[\cancel{\beta} \log \pi_{\theta}(\mathbf{y}|\mathbf{x}) - \cancel{\beta} \log \pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) - r(\mathbf{x}, \mathbf{y}) \right] \\ &= \arg \min_{\theta} \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \mathbb{E}_{\mathbf{y} \sim \pi_{\theta}(\cdot|\mathbf{x})} \left[\log \pi_{\theta}(\mathbf{y}|\mathbf{x}) - \left(\log \pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) + \frac{1}{\beta} r(\mathbf{x}, \mathbf{y}) \right) \right] \\ &= \arg \min_{\theta} \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \mathbb{E}_{\mathbf{y} \sim \pi_{\theta}(\cdot|\mathbf{x})} \left[\underbrace{\log \pi_{\theta}(\mathbf{y}|\mathbf{x})}_{\text{dependent on } \theta} - \underbrace{\log \pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) \exp \left(\frac{1}{\beta} r(\mathbf{x}, \mathbf{y}) \right)}_{\text{not dependent on } \theta} \right] \end{aligned}$$

equivalent

DPO Objective

$$\tilde{\theta} = \arg \min_{\theta} \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \mathbb{E}_{\mathbf{y} \sim \pi_{\theta}(\cdot|\mathbf{x})} \left[\underbrace{\log \pi_{\theta}(\mathbf{y}|\mathbf{x})}_{\substack{\text{dependent on } \theta \\ \Downarrow}} - \log \underbrace{\left(\pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) \exp \left(\frac{1}{\beta} r(\mathbf{x}, \mathbf{y}) \right) \right)}_{\text{not dependent on } \theta} \right] \checkmark$$

This equation defines the objective function as the difference between the log-probability distribution function of $\mathbf{y}$ and another function of $\mathbf{y}$.

This form of the objective function seems not “ideal”, as we usually prefer to see the difference between two distributions, so that we can interpret this difference as some kind of divergence between the distributions.

A simple idea is to convert the second term into a log-probability distribution over the domain of $\mathbf{y}$

DPO Objective

$$\tilde{\theta} = \arg \min_{\theta} \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \mathbb{E}_{\mathbf{y} \sim \pi_{\theta}(\cdot|\mathbf{x})} \left[\underbrace{\log \pi_{\theta}(\mathbf{y}|\mathbf{x})}_{\text{dependent on } \theta} - \underbrace{\log \left(\pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) \exp \left(\frac{1}{\beta} r(\mathbf{x}, \mathbf{y}) \right) \right)}_{\text{not dependent on } \theta} \right] \quad \begin{matrix} \times Z(\mathbf{x}) \\ \overline{Z(\mathbf{x})} \end{matrix}$$

See the second term as log of an unnormalized probability of $\mathbf{y}$, we can convert it into a normalized probability by dividing it by a normalization factor.

$$\underbrace{Z(\mathbf{x})}_{\text{norm}} = \sum_{\mathbf{y}} \underbrace{\pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) \exp \left(\frac{1}{\beta} r(\mathbf{x}, \mathbf{y}) \right)}$$

Hence we can define a probability distribution by

$$\underbrace{\pi^*(\mathbf{y}|\mathbf{x})} = \frac{\pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) \exp \left(\frac{1}{\beta} r(\mathbf{x}, \mathbf{y}) \right)}{Z(\mathbf{x})}$$

DPO Objective



$$\tilde{\theta} = \arg \min_{\theta} \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \mathbb{E}_{\mathbf{y} \sim \pi_{\theta}(\cdot|\mathbf{x})} \left[\underbrace{\log \pi_{\theta}(\mathbf{y}|\mathbf{x})}_{\text{dependent on } \theta} - \underbrace{\log \pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) \exp\left(\frac{1}{\beta} r(\mathbf{x}, \mathbf{y})\right)}_{\text{not dependent on } \theta} \right]$$

$$= \arg \min_{\theta} \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \mathbb{E}_{\mathbf{y} \sim \pi_{\theta}(\cdot|\mathbf{x})} \left[\log \pi_{\theta}(\mathbf{y}|\mathbf{x}) - \log \frac{\pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) \exp\left(\frac{1}{\beta} r(\mathbf{x}, \mathbf{y})\right)}{Z(\mathbf{x})} - \log Z(\mathbf{x}) \right]$$

*Rewriting
through $Z(\mathbf{x})$*

$$= \arg \min_{\theta} \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \mathbb{E}_{\mathbf{y} \sim \pi_{\theta}(\cdot|\mathbf{x})} \left[\log \pi_{\theta}(\mathbf{y}|\mathbf{x}) - \log \pi^{*}(\mathbf{y}|\mathbf{x}) - \log Z(\mathbf{x}) \right]$$

$$= \arg \min_{\theta} \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \left[\mathbb{E}_{\mathbf{y} \sim \pi_{\theta}(\cdot|\mathbf{x})} \left[\log \pi_{\theta}(\mathbf{y}|\mathbf{x}) - \log \pi^{*}(\mathbf{y}|\mathbf{x}) \right] - \mathbb{E}_{\mathbf{y} \sim \pi_{\theta}(\cdot|\mathbf{x})} [\log Z(\mathbf{x})] \right]$$

$$= \arg \min_{\theta} \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \left[\underbrace{\text{KL}(\pi_{\theta}(\cdot|\mathbf{x}) \parallel \pi^{*}(\cdot|\mathbf{x}))}_{\text{KL divergence}} - \underbrace{\log Z(\mathbf{x})}_{\text{constant wrt. } \theta} \right]$$

<https://arxiv.org/abs/2501.0922>

3

DPO Objective

$$\tilde{\theta} = \arg \min_{\theta} \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \left[\underbrace{\text{KL}(\pi_{\theta}(\cdot|\mathbf{x}) \parallel \pi^*(\cdot|\mathbf{x}))}_{\text{KL divergence}} - \underbrace{\log Z(\mathbf{x})}_{\text{constant wrt. } \theta} \right]$$

does not affect the result of the argmin operation, and can be removed from the objective.

$$\tilde{\theta} = \arg \min_{\theta} \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \left[\text{KL}(\pi_{\theta}(\cdot|\mathbf{x}) \parallel \pi^*(\cdot|\mathbf{x})) \right]$$

Clearly, the solution to this optimization problem is given by

$$\begin{aligned} \pi_{\theta}(\mathbf{y}|\mathbf{x}) &= \pi^*(\mathbf{y}|\mathbf{x}) \rightarrow \text{div.} = 0 \\ &= \frac{\pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) \exp\left(\frac{1}{\beta} r(\mathbf{x}, \mathbf{y})\right)}{Z(\mathbf{x})} \end{aligned}$$

DPO Objective

$$\begin{aligned}\pi_{\theta}(\mathbf{y}|\mathbf{x}) &= \pi^*(\mathbf{y}|\mathbf{x}) \\ &= \frac{\pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) \exp\left(\frac{1}{\beta} r(\mathbf{x}, \mathbf{y})\right)}{Z(\mathbf{x})}\end{aligned}$$

$$\exp\left(\frac{1}{\beta} r(\mathbf{x}, \mathbf{y})\right) = \frac{\pi_{\theta}(\mathbf{y}|\mathbf{x})}{\pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x})} \cdot \log[Z(\mathbf{x})]$$

Given this equation, we can express the reward $r(\mathbf{x}, \mathbf{y})$ as

$$r(\mathbf{x}, \mathbf{y}) = \beta \left(\log \frac{\pi_{\theta}(\mathbf{y}|\mathbf{x})}{\pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x})} + \log Z(\mathbf{x}) \right)$$

Interesting because we eventually obtain a representation of the reward model based on the policy

we can apply it to the Bradley-Terry model to calculate the preference probability

DPO Objective

$$\underline{r(\mathbf{x}, \mathbf{y})} = \underline{\beta \left(\log \frac{\pi_{\theta}(\mathbf{y}|\mathbf{x})}{\pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x})} + \log Z(\mathbf{x}) \right)}$$

$$\begin{aligned} \Pr_{\theta}(\mathbf{y}_a \succ \mathbf{y}_b | \mathbf{x}) &= \text{Sigmoid}(r(\mathbf{x}, \mathbf{y}_a) - r(\mathbf{x}, \mathbf{y}_b)) \\ &= \text{Sigmoid} \left(\beta \left(\log \frac{\pi_{\theta}(\mathbf{y}_a | \mathbf{x})}{\pi_{\theta_{\text{ref}}}(\mathbf{y}_a | \mathbf{x})} + \log Z(\mathbf{x}) \right) - \right. \\ &\quad \left. \beta \left(\log \frac{\pi_{\theta}(\mathbf{y}_b | \mathbf{x})}{\pi_{\theta_{\text{ref}}}(\mathbf{y}_b | \mathbf{x})} + \log Z(\mathbf{x}) \right) \right) \\ &= \text{Sigmoid} \left(\beta \log \frac{\pi_{\theta}(\mathbf{y}_a | \mathbf{x})}{\pi_{\theta_{\text{ref}}}(\mathbf{y}_a | \mathbf{x})} - \beta \log \frac{\pi_{\theta}(\mathbf{y}_b | \mathbf{x})}{\pi_{\theta_{\text{ref}}}(\mathbf{y}_b | \mathbf{x})} \right) \end{aligned}$$

Handwritten notes: Red arrows point from $\mathbf{y}_a$ and $\mathbf{y}_b$ in the first line to the corresponding terms in the final line. The word "wimpy" is written in red next to the arrow from $\mathbf{y}_a$, and "why" is written in red next to the arrow from $\mathbf{y}_b$.

DPO Objective

$$\Pr_{\theta}(\mathbf{y}_a \succ \mathbf{y}_b | \mathbf{x}) = \text{Sigmoid} \left(\beta \log \frac{\pi_{\theta}(\mathbf{y}_a | \mathbf{x})}{\pi_{\theta_{\text{ref}}}(\mathbf{y}_a | \mathbf{x})} - \beta \log \frac{\pi_{\theta}(\mathbf{y}_b | \mathbf{x})}{\pi_{\theta_{\text{ref}}}(\mathbf{y}_b | \mathbf{x})} \right)$$

desired $\uparrow$ high-reward
or high-reward (not-desired)

This formula is elegant because it converts the difference in rewards into the difference in ratio functions, and we do not need to calculate the value of $Z(\mathbf{x})$.

Finally, we can train the target policy by minimizing the following DPO loss function $\uparrow$

$$\mathcal{L}_{\text{dpo}}(\theta) = -\mathbb{E}_{(\mathbf{x}, \mathbf{y}_a, \mathbf{y}_b) \sim \mathcal{D}_r} [\log \Pr_{\theta}(\mathbf{y}_a \succ \mathbf{y}_b | \mathbf{x})]$$

$$\pi_{\theta}(\mathbf{y}_{\text{loss}} | \mathbf{x}) < \pi_{\theta_{\text{ref}}}(\mathbf{y}_{\text{loss}} | \mathbf{x})$$

It should be noted that the loss function here depends on the parameters of the policy (i.e., θ) rather than the parameters of the reward model

$$\pi_{\theta}(\mathbf{y}_{\text{win}} | \mathbf{x}) > \pi_{\theta_{\text{ref}}}(\mathbf{y}_{\text{win}} | \mathbf{x})$$

DPO Variant: KTO [<https://arxiv.org/abs/2402.01306>]

Implements a strategy that utilizes a single preference

$$\mathcal{L}_{\text{KTO}}(\pi_{\theta}, \pi_{\text{ref}}; \beta) = \mathbb{E}_{x, y \sim \mathcal{D}} \left[1 - \hat{h}(x, y; \beta) \right]$$

$$\hat{h}(x, y; \beta) = \begin{cases} \sigma \left(\beta \log \frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)} - \mathbb{E}_{x' \sim \mathcal{D}} [\beta \text{KL}(\pi_{\theta} \parallel \pi_{\text{ref}})] \right) & \text{if } y \sim y_{\text{desirable}} | x, \\ \sigma \left(\mathbb{E}_{x' \sim \mathcal{D}} [\beta \text{KL}(\pi_{\theta} \parallel \pi_{\text{ref}})] - \beta \log \frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)} \right) & \text{if } y \sim y_{\text{undesirable}} | x \end{cases}$$

π_{ref} $\rightarrow$ user pool π_{θ}

prompt
winning, $\hookrightarrow$ lossy

DPO: Advantage

- The main advantage of DPO lies in its simplicity and efficiency.
- The DPO objective is very straightforward —it directly optimizes for preference-based feedback, rather than relying on separately developed reward models.
- Moreover, DPO is generally more sample-efficient, as it learns from a fixed dataset without the need for the computationally expensive sampling process used in PPO.
- This makes DPO a popular method for human preference alignment, especially when developing and applying reward models via reinforcement learning is challenging.

DPO can broadly be viewed as an offline reinforcement learning method

PPO: Advantage

- Online reinforcement learning methods like PPO, which require exploring new states through interaction with the environment (using the reward model as a proxy), also have their unique advantages.
- One of the benefits of online reinforcement learning is that it allows the agent to continuously adapt to changes in the environment by learning from real-time feedback.
- This means that, unlike offline methods, online methods are not constrained by the static nature of pre-collected data and can discover new problem-solving strategies.
- In addition, **exploration** can help the agent cover a wider range of state-action pairs, thus improving generalization.
- This could be an important advantage for LLMs, as generalization is considered a critical aspect in applying such large models



Inference-time alignment

One of the significant limitations of many such methods is that LLMs must be fine-tuned. For RLHF and its variants, training LLMs with reward models can be computationally expensive and unstable, leading to increased complexity and costs when applying these approaches.

What if we have a reward model but do not want to train RL policy? Can we still use it at inference time?

Inference-time alignment: BoN sampling

Use the reward model to select the best one from N alternative outputs generated by the LLM, a method known as Best-of- N sampling (BoN sampling)

The LLM takes the input sequence x and generates N different output sequences

$$\{\hat{y}_1, \dots, \hat{y}_N\} = \underset{y}{\operatorname{argTopN}} [\operatorname{Pr}(y|x)]$$

The reward model is used to evaluate and select the best one

$$\hat{y}_{\text{best}} = \max \{ \underset{r(\mathbf{x}, \hat{y}_1)}{\quad}, \dots, \underset{r(\mathbf{x}, \hat{y}_N)}{\quad} \}$$

BoN sampling can also be used to train LMs

We first select the “best” outputs from the N-best lists via the reward model, and then take these selected outputs to fine-tune the LLM.

In this way, we can introduce human preferences into the training of LLMs via a much simpler approach compared to RLHF.

Also known as “*rejection sampling*”

REFERENCES

Foundations of Large Language Models:
<https://arxiv.org/abs/2501.09223>



THANK YOU